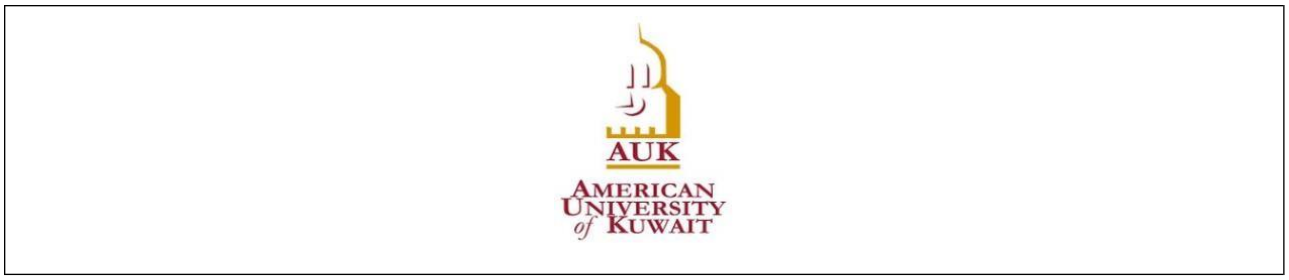


A Capstone Project Submitted to the Faculty of the

Computer Science and Information Systems Program

College of Engineering and Applied Sciences

American University of Kuwait

In Partial Fulfillment of the Requirements for the degree

Bachelor of Science in [Computer Science/Information Systems]

By

Zahraa Khalaf, 58128

Farah Al-Bashiti, 65821

Noor Alzubaidi, 56262

Eman Aldossari, 53895

SignSpeak Software

Advisor

Aaron Rasheed Rababaah

Course Coordinator

Shereef Almaati

Jan 4

Fall, 2025-2026

Computer Science and Information Systems Program

College of Engineering & Applied Sciences

American University of Kuwait

Approval Sheet – CSIS 490 Capstone I

The Capstone Project entitles **SignSpeak** prepared and submitted by **Zahraa Khalaf, Farah Al-Bashiti, Noor Alzubaidi, & Eman AlDossari (58128, 65821, 56262, 53895),** has been examined and is recommended for approval and acceptance

Recommended: Date:

_____ _____

(Signature of Advisor over printed name)

Approved by the Oral Defense Panel with a grade **[Specify the Grade]**

Approved: Date:

_____ _____

(Signature of the Panel Chair over printed name)

Accepted and approved in partial fulfillment of the requirements in Bachelor of Science in [Computer Science/Information Systems]

Approved: Date:

_____ _____

Dedication

We dedicate this project to individuals with speech impairments and their families. Your experiences have been a source of inspiration and a reminder of the importance of inclusive and accessible technology. We also acknowledge the professionals and advocates who dedicate their efforts to supporting and improving the lives of the speaking-impaired community. We would like to express our sincere appreciation to our families for their consistent support and for providing us with valuable insights that contributed to the development of this project. We are also grateful to our supervisor for their guidance, encouragement, and the resources they provided throughout the process. Lastly, we extend our thanks to all the professors who have contributed to our learning and growth. Your dedication and commitment to our education have played a vital role in helping us reach this point in our academic journey.

Acknowledgement

The progress and achievements of our Capstone 1 project reflect the dedication, collaboration, and support we received from multiple people throughout our journey. We extend our most sincere appreciation and thanks to Dr. Aaron Rasheed Rababaah, our capstone advisor, whose expertise in machine learning and deep learning played a crucial role in shaping the foundation of SignSpeak. His openness, feedback, and availability immensely impacted our development process and our confidence as a team, and without his help, many of the challenges we faced would have been far more difficult to overcome. His ability to guide us through everything made all the difference. We would also like to thank Dr. Shereef Almaati, our capstone course coordinator, for creating a structured environment that helped us stay organized

and follow the steps needed to reach completion. A special thanks goes to Dr. Mohammed El Abd, Dean of the College of Engineering and Applied Sciences, for making it possible for us to enroll in Capstone 1 during the spring and for his encouragement and support we needed throughout the process.

To our team members, Eman AlDossari, Farah Al-Bashiti, Noor Alzubaidi, and Zahraa Khalaf, thank you for being more than just colleagues. Your determination helped turn our ideas into reality. From not only solving problems together but also supporting each other emotionally, this project was a shared effort between us all, where each of us brought our own unique touch, combining them into something bigger. Your dedication and leadership will always be something memorable and truly appreciated.

We are grateful to Al-Amal School for Girls, who welcomed us, shared their knowledge and insights, and guided us in understanding the needs of the Arabic Sign Language community. Their feedback and participation really helped us shape SignSpeak into something practical and meaningful.

Lastly, thank you to our families, friends, and the American University of Kuwait for always being there, whether it was through encouragement or simply by believing in us. Your support made this journey more fulfilling.

Abstract

SignSpeak is a sign language translation software developed in Kuwait to enhance communication between Arabic Sign Language (ArSL) users and individuals who do not understand sign language. The project targets the communication barriers faced by deaf and mute communities by providing a real-time system that converts hand gestures into both text and speech. Leveraging computer vision and machine learning, SignSpeak accurately detects and translates hand gestures, promoting inclusivity and accessibility.

Tailored for Kuwait's linguistic and cultural context, SignSpeak provides a practical and localized solution for inclusive communication. Developed in Python and optimized for camera-enabled devices, the software aims to support educational, social, and professional engagement for Arabic Sign Language users across the country.

The software is designed not only for individuals with hearing or speech impairments but also for non-sign language users—including family members, educators, medical staff, and service providers—enabling effective communication without the need for an interpreter. A built-in recording feature allows users to save translated interactions for review or learning. Future updates will include video-based modules to support Arabic Sign Language education.

Table of Contents

LIST OF TABLES

LIST OF FIGURES

Work Distribution Table

Member	Contribution
Farah Al-Bashiti	- **Chapter 1:** Project Context, Supervised: Project Objectives - **Chapter 2:** Issues with Prior Projects, Supervised: Related Projects - **Chapter 3:** How the Project will work - **Chapter 4:** Operational Feasibility (Fishbone Diagram) - Schedule Feasibility (GANTT Chart) - Data and Process Modelling (System Flowchart) - Object Modelling (Use Case Diagram, Activity Diagram, High-Level Class Diagram) - Risk Assessment/Analysis - **Chapter 5: Detailed OO** Design (Detailed Class Diagram, Sequence Diagram) - Development (Hardware Specification) - Brief Manual - Verification, Validation Testing (Unit Testing, Compatibility Testing) - Recommendations
Zahraa Khalaf	- **Chapter 1:** Project Objectives - **Chapter 2:** Related Literature - **Chapter 3:** Technicality of the Project, Details of the technologies - **Chapter 4:** Technical Feasibility (Relevance of the technologies) - Relevance of the technologies - Requirements Modelling (Process, Performance) - Data and Process Modelling (Program Flowchart) - **Chapter 5:** Output and User Interface Design (Reports)

	- Brief Manual Checking - Verification, Validation Testing (Performance Testing, Load Testing, System Testing)
Noor Al-Zubaidi	- **Chapter 1:** Purpose and Description of the Project, Significance of the Project - **Chapter 2:** Theoretical Background - **Chapter 4:** Operational Feasibility (Functional Decomposition Diagram) - Technical Feasibility: Compatibility checking (hardware/software and other technologies) - Requirements Modelling (Input, Output, Control) - Object Modelling (High -LevelClass Diagram) - **Chapter 5:** Output and User Interface Design (Forms) - Data Design (Data Dictionary) - Development (Software Specification) - Glossary
Eman Al-Dossari	- Table of Figures, Table of Tables - Chapter 1: Survey - **Chapter 2:** Related Projects - **Chapter 4:** Economic Feasibility (Cost and Benefit Analysis, Cost Recovery Scheme) - Data and Process Modelling (Context Diagram, Data Flow Diagram)

	- **Chapter 5:** Data Design (Entity Relationship Diagram) - System Architecture (Network Model, Network Topology, Security) - Development (Program Specification) - Test Plan - Verification, Validation Testing (Stress Testing) - Conclusion - System Impact

Table 1: Work Distribution

Chapter 1: Introduction

Introduction Proper

Project Context

In today's increasingly digital and interconnected world, effective communication is a fundamental necessity. However, for individuals who are **deaf or mute**, especially those relying on **Arabic Sign Language (ArSL)**, communication with the general public can still be significantly limited. This communication barrier is particularly noticeable in countries like **Kuwait**, where awareness and availability of inclusive technologies remain limited.

While sign language serves as a vital tool for expression among these communities, it is not widely understood by the general population, including **educators, healthcare providers, service personnel, and even family members**. As a result, many Arabic Sign Language users in Kuwait face challenges in accessing essential services, participating in educational or professional settings, and engaging socially.

SignSpeak emerges from this need, designed as an assistive software solution that translates Arabic Sign Language into text and speech in real-time. The project leverages machine learning and computer vision to detect hand gestures and interpret them accurately, providing an essential bridge between sign language users and the hearing-speaking population.

Moreover, by being developed locally in Kuwait, SignSpeak is tailored to the cultural and linguistic nuances of Arabic Sign Language as used in the region. This ensures greater accuracy and usability for the target population. The software also includes a **recording feature** to allow

deaf/mute users to record themselves, and their gestures will be translated immediately, or by the speaking/hearing users to record the sign language speaker and understand them in real-time.

Ultimately, SignSpeak aims to promote **accessibility, inclusion, and empowerment** for individuals with communication challenges while fostering a more understanding and connected society in Kuwait.

Survey Results

Before moving on to the purpose, it was key that we listen to our audience and future users on how they feel about ASL translation through an app, which made us survey 83 individuals with the following results:

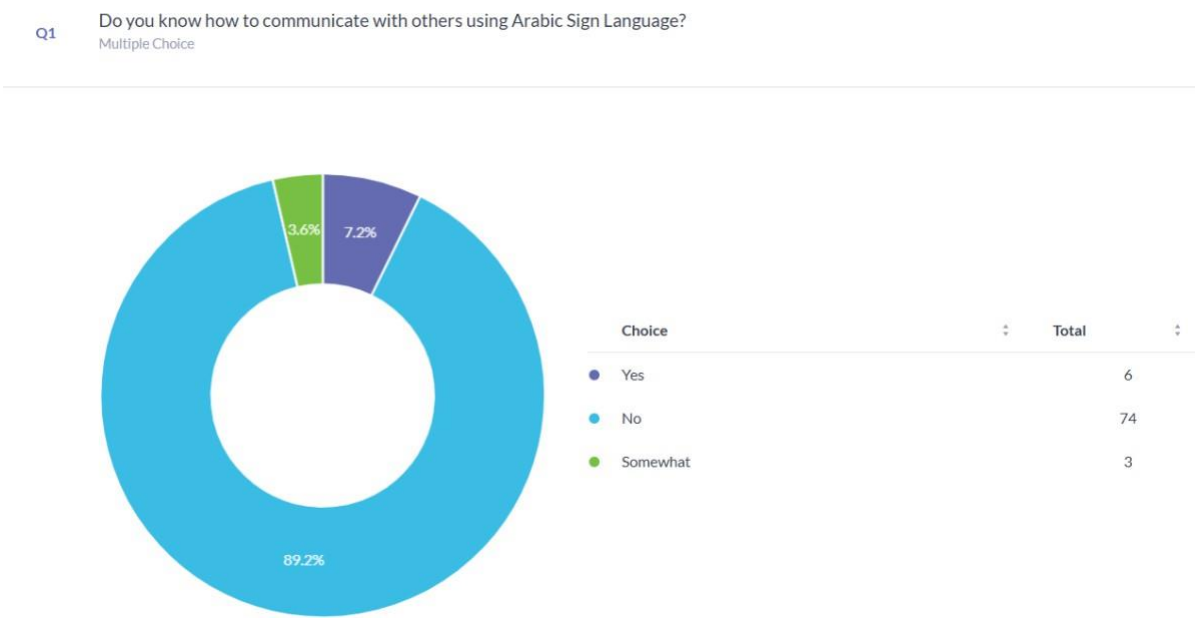
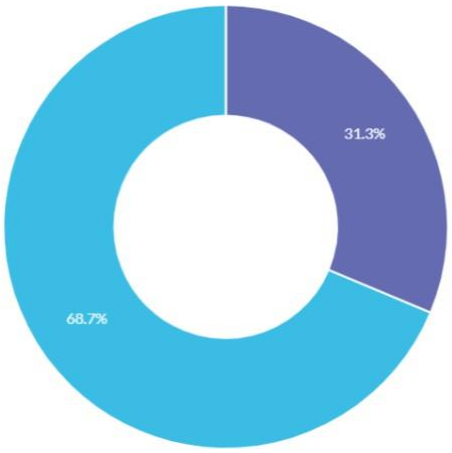


Figure 1: Survey Results 1

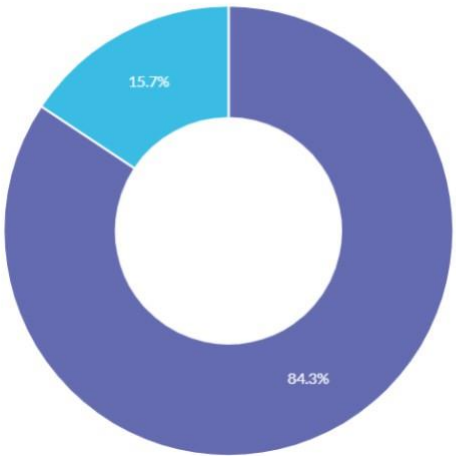
Q2 Do you know a person who is deaf / hard of hearing / struggling with speech?
Multiple Choice



Choice	Total
Yes, I do	26
No	57

Figure 2: Survey Results 2

Q3 If there was a software used for translating sign language into speech and text, would you use it?
Multiple Choice



Choice	Total
Definitely!	70
No, I won't need such help	13

Figure 3: Survey Results 3

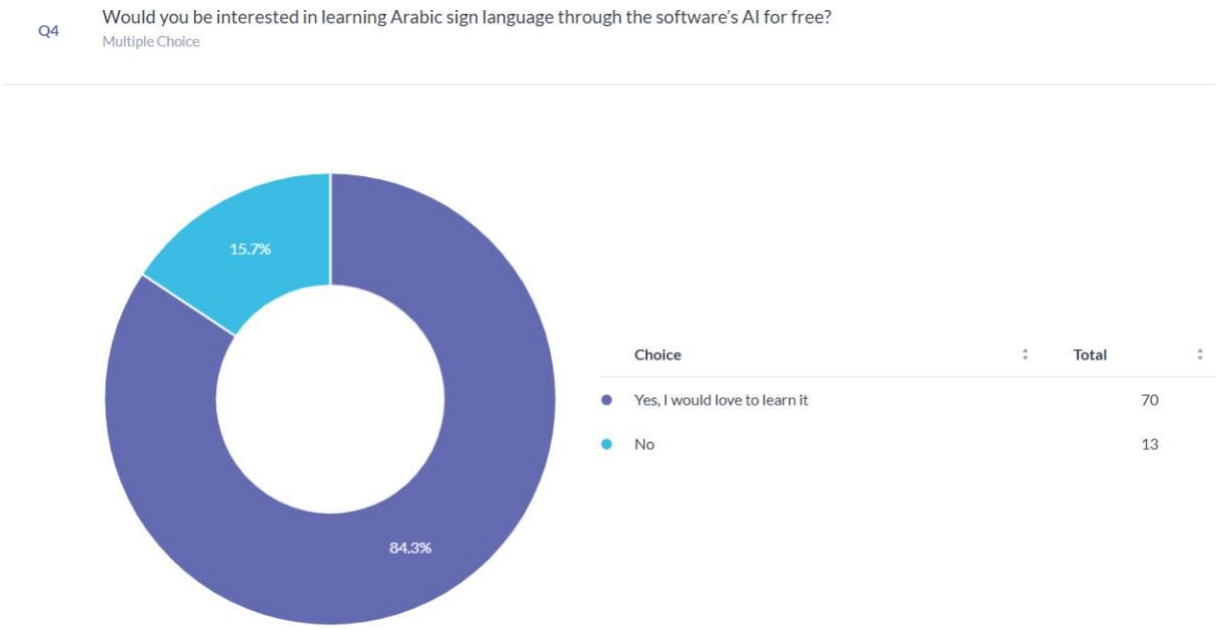


Figure 4: Survey Results 4

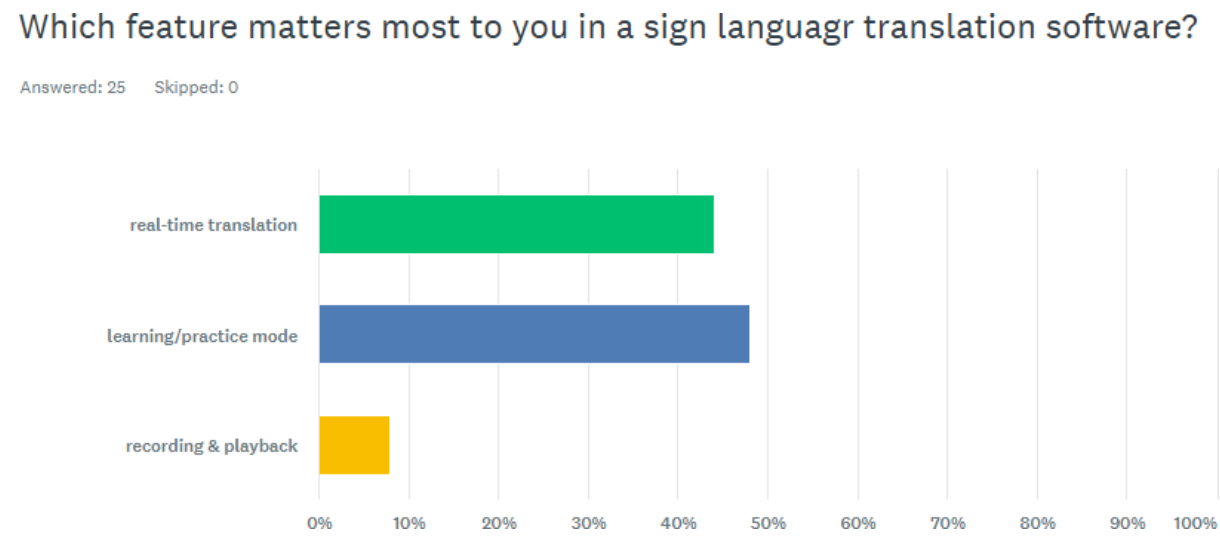


Figure 5: survey results

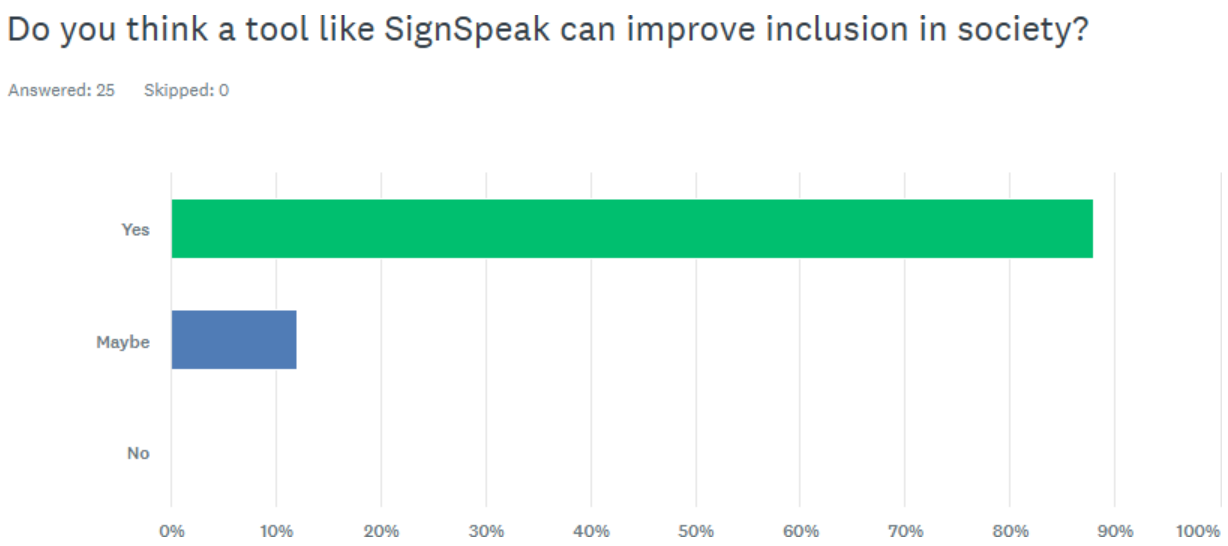


Figure c: survey results

Purpose and Description of the Project

The primary purpose of SignSpeak is to enable communication between Arabic Sign Language (ArSL) users and individuals who may not understand it. The software's initial focus is to translate ArSL hand gestures into both text and speech outputs, which is done through capturing the individuals' hand movements through a camera and processing the gestures to generate accurate and real-time translations.

To accomplish this, SignSpeak will integrate machine learning, computer vision, and gesture detection libraries to accurately recognize and interpret hand movements into the corresponding meaning. The software will be designed with a user-friendly graphical interface developed in Python, and mobile versions are planned to use React Native or Flutter.

If time permits, the software will incorporate additional features, such as the practice mode, that is planned to provide educational value. This feature would allow users to upload recorded videos, receive practice prompts, and get automated feedback based on their signing accuracy, ultimately helping them practice and improve their ArSL proficiency.

Project Objectives

Our mission is to empower the speaking- and hearing-impaired community in Kuwait and the broader MENA region by providing accessible, AI-driven communication tools. This software enables users to convey their messages independently, reducing reliance on human interpreters. In the future, we aim to introduce educational services that include interactive sign language learning. Through video-based lessons, users will be prompted to perform sign gestures, which the AI will evaluate for accuracy. This feature will not only enhance learning but also promote broader awareness and understanding of sign language.

Significance of the Project

SignSpeak addresses an important yet overlooked need in the field of assistive communication technology. While several other tools exist for American Sign Language (ASL), such as SignAll or HandSpeak, these systems are either primarily focused on only ASL, require expensive hardware, or are not publicly available, making them highly inaccessible to the average individual. Currently, there is no widely accessible software designed specifically for Arabic Sign Language users in Kuwait or the MENA region.

This is where SignSpeak provides a valuable solution, filling a significant gap in the field due to the lack of software dedicated specifically to the MENA regions where Arabic Sign Language is used. It serves as an accessible tool with the ability to interpret ArSL hand gestures and translate them into both text and speech, creating a more inclusive communication environment.

Ultimately, SignSpeak is more than just a communication aid. It becomes a platform for raising awareness about the communication challenges faced by ArSL users, promoting accessibility and inclusivity across social and professional settings, and encouraging and inspiring innovation in assistive technology within Kuwait and the broader MENA region. It leaves a positive impact not only on the individual that struggles with speech impairments but also on their interactions with their families, equipping education facilities, and overall broadening our society's understanding and support for the deaf and mute communities.

Chapter 2: Review of Related Literature

Theoretical Background

Speech is regarded as one of the most fundamental ways people communicate; it is efficient and allows us to exchange information and thoughts with each other through an auditory medium. However, not everybody can rely on it. For individuals who are deaf or mute, speech can become a barrier to their daily lives, affecting their social environment, education, and careers [1]. These limitations motivated research into augmentative and alternative communication (AAC) systems. AAC systems are an integrated group of components and strategies such as symbols and gestures, dedicated to assisting individuals with communication disorders [2]. Sign language is an unaided form of AAC, as it is not a substitute for speech, but instead it is its own language with its own syntax and grammar [2], [3]. Similar to speech, it allows us to converse with each other but instead, it is done visually, by combining hand gestures, body language, and facial expressions to convey meaning and express emotions [3].

Since sign language is a visual language that depends on gestures and hand movements, gesture recognition within human-computer interaction (HCI) has become an important aspect in technology. It helps us understand how different individuals may interact with different systems and allows us to take that input and feedback and turn it into a system design that is intuitive and accessible. HCI allows us to interact with these systems through body motion. By capturing the user's motions through a camera, it can process and extract specific features, such as hand and finger positioning and movement. After extracting, it then matches the features to another database of known movements and gestures to identify the most likely result [4]. To achieve this, most systems use convolutional neural networks (CNNs), a type of machine learning model that

analyzes visual data [5]. They identify patterns such as edges and curves, as well as spatial structures in images and video frames, and can help gesture recognition by identifying static hand positions and tracking how they move. They can also detect changes in visual inputs, which are necessary for accuracy and for differentiating between gestures that may look alike [5]. However, these features come at the cost of requiring large datasets that are of higher quality and have been trained proficiently. This heavily impacts the results of many sign languages that are not commonly used, such as Arabic Sign Language (ArSL), due to them being underrepresented in the field and the lack of availability of proper datasets for them [6], [7]. Fortunately, work around can be done by using transfer learning, which is acquiring a model that has already been trained with the needed resources and adjusting it to fit a more niche and specific dataset, such as the Arabic Sign Language one [8].

Moreover, for CNN to function effectively, the data that is inputted is required to be accurate and precise, and so it is necessary that we use computer vision tools to detect hand landmarks in real-time [9]. Libraries such as OpenCV and MediaPipe can be used to pinpoint key features of the hand that are being tracked, such as the wrists, the fingers, and the knuckles, while only using a camera [9]. This allows for more accessibility, as they can run on any device that has a built-in camera and removes the need for any expensive or uncomfortable requirements such as gloves or sensors. However, any changes in the lighting or the background, as well as any other hand entering the view of the camera, can disrupt the detection process and lead to errors [9]. After a gesture has been recorded and matched correctly, it still needs to be communicated to someone who may not understand it [2]. Text-to-speech (TTS) allows any text to be converted into an audio format, and many modern TTS models, such as FastSpeech2, make use of deep learning to exhibit human-like voices by generating speech from the text input in real-time, making it faster

and more flexible [10]. Within AAC systems, it can behave as the voice of the people whose primary language is sign language.

Related Literature

In recent years, the advancement of artificial intelligence and computer vision has enabled new possibilities for supporting individuals with hearing and speech impairments. Among these innovations, systems that translate sign language into spoken language and text have become a growing area of research and development. These technologies aim to bridge communication gaps between the hearing and non-hearing communities, improving accessibility and social inclusion. This literature review explores existing works and technologies related to sign language recognition and translation. It examines current methods used for gesture detection, natural language processing, and speech synthesis, as well as the strengths and limitations of available systems. By analyzing previous research, this review identifies the challenges in building accurate, real-time, and regionally adapted solutions, particularly those relevant to Arabic sign language and users' needs in Kuwait and the MENA region. This foundation supports the development of our system, which aims to provide an AI-powered, user-friendly platform for translating sign language into both text and spoken language.

In the study titled *"**Arabic Sign Language Recognition using Deep Learning: A Comparative Study of CNN and Transformer Models,"*** the authors [11] investigate the use of convolutional neural networks (CNNs), transformer-based models, and classification techniques for input processing and feature extraction to recognize individual Arabic sign language letters. The study utilizes two key datasets: the ArSL2018 dataset [12] and the RGB Arabic Alphabets Sign Language dataset (ArASL) [13]. The recognition system proposed in the study consists of

three steps: data pre-processing, model selection with transfer learning, and model evaluation [11]. They started with standardizing the data to get optimal training for deep learning models [11]. Then, they leveraged both Convolutional Neural Network (CNN) architectures and transformer-based models, employed fine-tuning strategies, froze the early layers of these models, and replaced them with final layers configured for the Arabic 28-class alphabet recognition [11]. The final results show that transfer models outperform transformer-based models in terms of training efficiency, while transformer models had more test accuracy and more training time [11]. While the study provides valuable datasets and effective training strategies for CNN and transformer models, it remains limited to static alphabet recognition. In contrast, our project expands this scope by focusing on real-time recognition of complete words and phrases, addressing more practical communication needs.

This study presents a translation system that converts Arabic text into Arabic Sign Language (ArSL) [14]. The authors developed a custom corpus of 203 signed sentences with 710 distinct signs, explicitly focusing on instructional language commonly used in deaf education. Despite this tailored corpus, the system achieved a relatively high Word Error Rate (WER) of 46.7% and a Position-independent Error Rate (PER) of 29.4%, indicating challenges in accurately generating sign language output [14]. Instead of traditional statistical or deep learning models, the authors employed a chunk-based Example-Based Machine Translation (EBMT) approach due to the limited size of the available corpus. This method retrieves pre-aligned text-sign chunks from a database and recombines them to generate the final output [14]. The study also provides foundational insights into the linguistic structure of ArSL. It highlights the importance of manual features (MFs) such as hand shape, hand position relative to the body, and direction of movement and non-manual features (NMFs), which include facial expressions,

shoulder movements, and head tilts that co-occur with MFs [14]. These components play a critical role in the semantic accuracy of sign production, as missing or incorrect non-manual features can significantly alter the intended meaning. Importantly, the study debunks a common misconception by clarifying that ArSL is an independent natural language with its grammar and structure rather than a visual representation or fingerspelling of Arabic. Fingerspelling is only used in limited contexts, such as for proper names or terms without established signs [14]. While the system offers valuable contributions to understanding ArSL's linguistic features and provides a working prototype, the reliance on a small corpus and the lack of model training severely limits its performance. Future improvements could involve incorporating larger annotated datasets and training data-driven machine learning models, such as neural networks, to enhance translation quality and generalization capabilities.

Bani Baker et al. [15] proposed a robust Arabic Sign Language (ArSL) recognition system using convolutional neural networks (CNNs) combined with state-of-the-art transfer learning techniques. The study utilized the Arabic alphabet's Sign Language Dataset (ArASL), which includes 54,049 labeled images representing 32 classes contributed by over 40 individuals [15]. To address class imbalance, a fixed number of samples was allocated per class [15]. Preprocessing steps included resizing all images to a standard resolution of 64×64 pixels, normalization to standardize visual features, and applying data augmentation techniques such as rescaling, zooming, flipping, and shifting to enhance model generalization and prevent overfitting [15]. Six pre-trained models, MobileNetV2, VGG16, InceptionV3, ResNet50V2, ResNet152, and Xception, were fine-tuned using ImageNet weights, with a softmax prediction layer appended after the last fully connected layer [15]. Among them, ResNet50V2 and InceptionV3 achieved the highest performance, reaching 100% accuracy with 0% error,

supported using early stopping and adaptive learning rates [15]. These findings highlight the effectiveness of using transfer learning for ArSL recognition and underscore the robustness of ResNet50V2 and InceptionV3 in avoiding overfitting while maintaining superior accuracy [15].

Related Projects

We chose to begin with mentioning Handspeak for this area of our writing; it is a visual dictionary for American Sign Language (only), and it differs from our project as it is not a real-time translation system/software. Similar to our software, Handspeak's goal is to bridge the communication gap between the two groups: individuals who use sign language and those who don't use it. They provide users with videos that teach words, numbers, and phrases in ASL. Handspeak also tackles the grammar of ASL, which is very interesting and useful. The website explains in detail how a person's facial expression can convey different meanings.

The Eshara application, developed by the Ministry of Education in Saudi Arabia, is a key initiative designed to bridge communication gaps for individuals with hearing impairments. This innovative digital platform aims to enhance accessibility and independence for deaf individuals when interacting with both government and private services [17]. Eshara facilitates real-time visual communication between deaf clients and specialized call centers for remote sign language translation. A sign language translator acts as an intermediary, converting sign language into spoken Arabic and vice versa, ensuring seamless communication between the deaf client and the service provider. This eliminates communication barriers and streamlines procedures for deaf individuals. It operates on a foundation of key values: customer satisfaction, creativity, flexibility, trust, and secrecy, ensuring a reliable and user-centric experience [17].

An upcoming product, rather than a project, is SignAll, software developed by a company that aims to leverage AI and camera vision to analyze and translate ASL. It is marketed as intuitive, lightning-fast, accessible, and accurate [18]. Two strength points that SignAll holds are that it supports multimedia uploading and is built on a cloud infrastructure. Users can send an image, a prerecorded video, or a real-time one that can all be translated. The access to the dataset is fast, with accurate translation time due to their use of cloud computing.

Issues with Prior Projects

Although new channels for communication and accessibility have been made possible by developments in sign language technology, the usefulness and efficacy of current systems are still severely limited. Although programs like HandSpeak and SignAll have been helpful in advancing sign language learning and translation, they are unable to meet the ever-changing needs of interactive, real-time communication.

The main purpose of HandSpeak is to teach individual sign demonstrations using still images and short videos. This structure makes it well-suited for foundational learning but inherently limits its capacity to support live, two-way communication. Without features like gesture tracking, adaptive feedback, or context-aware recognition, users are unable to engage in meaningful conversations, reducing the tool's utility in real-world interaction scenarios. Additionally, the platform lacks interactive elements such as quizzes or progress tracking, which could enhance user engagement and learning outcomes [19].

SignAll, on the other hand, incorporates advanced machine vision techniques for real-time ASL translation. However, it is still in a pre-release stage, with access limited to those who join the waiting list. Moreover, the system's dependence on multiple external sensors and calibrated

environments poses challenges for widespread adoption. These hardware requirements introduce barriers related to cost, portability, and compatibility with common devices—factors that restrict accessibility for many potential users. Furthermore, the necessity for a complex setup, including multiple cameras and sensors may deter casual users from seeking a more straightforward solution [20].

Another critical concern lies in gesture recognition accuracy, which remains vulnerable to performance drops under inconsistent lighting, varied skin tones, or diverse hand shapes. The generalizability of existing models is limited, especially when trained on narrow or homogeneous datasets, reducing their effectiveness across a broad user base. Incorporating more diverse training data and enhancing algorithms to better handle environmental variations could significantly improve system reliability. For instance, a study by Hosain et al. highlights the challenges in achieving accurate hand shape recognition under varying conditions and emphasizes the need for robust models that can generalize across different users and environments [21].

In conclusion, while current sign language technologies reflect important progress, they often lack the versatility and robustness needed for spontaneous, inclusive, and accessible communication. Enhancing model adaptability, simplifying hardware dependencies, and ensuring equitable access are essential steps toward creating more impactful and scalable solutions.

Chapter 3: Technical Background

Technicality of the Project

This section provides an overview of the key technologies, tools, and methods used in building the project. It outlines the software and technical processes involved in developing the system, from data handling and model integration to real-time functionality. Further details on each component will be discussed in the following subsections.

Details of the Technologies to be Used

Development Tools

- **Datasets**:
  - **Primary:** our own collected gestures dataset of 34 signs.
- **Deep Learning Frameworks**:
  - **tensorflow:** for tunning and training the gesture recognition model.
- **Computer Vision**:
  - **OpenCV**: for accessing the webcam and tracking hand gestures in real time.
  - **MediaPipe:** for easier hand landmark detection and gesture feature extraction.
- **Real-Time Processing**:
  - **Time library**: to manage delays, frame rates, and timing control during gesture capture.
- **Text-to-Speech Conversion:**
  - **gTTs**: for converting recognized gestures into audible Arabic speech output.
- **For Displaying Text:**
  - **Arabic-reshaper:** for displaying Arabic text.

Software Environment

- **Programming Language**: Python.
- **Integrated Development Environment (IDE)**: PyCharm and jupyter notebook.
- **Dependencies**: TensorFlow, MediaPipe, OpenCV, gTTs, NumPy, etc.

Hardware Requirements

Table 2: Hardware requirements table

Component	Minimum Requirement	Purpose / Description
CPU	Intel Core i5 / AMD Ryzen 3 or better	Handles model inference, GUI rendering, and webcam data processing.
RAM	8 GB minimum	Supports TensorFlow, MediaPipe, and GUI operations simultaneously.
Storage	At least 10 GB of free space	Stores datasets, model files, and software dependencies.
GPU	Not required for inference	Optional for accelerating training and tuning.
Webcam	Standard HD (720p or higher)	Captures hand gestures in real-time.
Operating System	Windows, macOS, or Linux	Ensures cross-platform compatibility.
Internet Connection	Required only during initial setup	Used to download models, datasets, and packages.

How the Project will Work

Functionality 1: Translating Arabic Sign Language gestures into text and speech

This is the core functionality of the SignSpeak software. The system will use a front-facing camera (e.g., webcam or mobile camera) to continuously capture hand gestures performed by the user. A deep learning-based gesture recognition model will analyze the gestures and compare them to a trained dataset of Arabic Sign Language (ArSL). Once a gesture is recognized, the corresponding Arabic word or phrase will be displayed as text on the screen. Immediately after, a text-to-speech (TTS) engine will vocalize the translated text using Arabic audio output, enabling communication with hearing individuals. The system is optimized for real-time processing to reduce latency and maintain fluent interaction.

Functionality 2: Real-time recording of translated conversations

This functionality allows users to save their translated sign-to-text/speech sessions for future reference or study. As the user performs gestures and the software translates them into text and speech, the session can be recorded manually. The recorded data will include the recognized sign, corresponding text, and audio output, stored in a structured format (like .mp3). This feature will be especially useful for replaying important conversations or reviewing commonly used signs.

Functionality 3: Educational Model

This functionality provides an educational model that allows users to learn and practice Arabic Sign Language directly within the software. It is triggered by pressing a specific keyboard key (“L”), that activates a learning form that overlays an instructional video or a 3D model of a specific sign onto the user’s live camera feed. This allows user’s to mimic the movements of the sign in real-time while simultaneously viewing their own gestures.

Chapter 4: Methodology

Requirements Specifications

Operational Feasibility

Fishbone Diagram

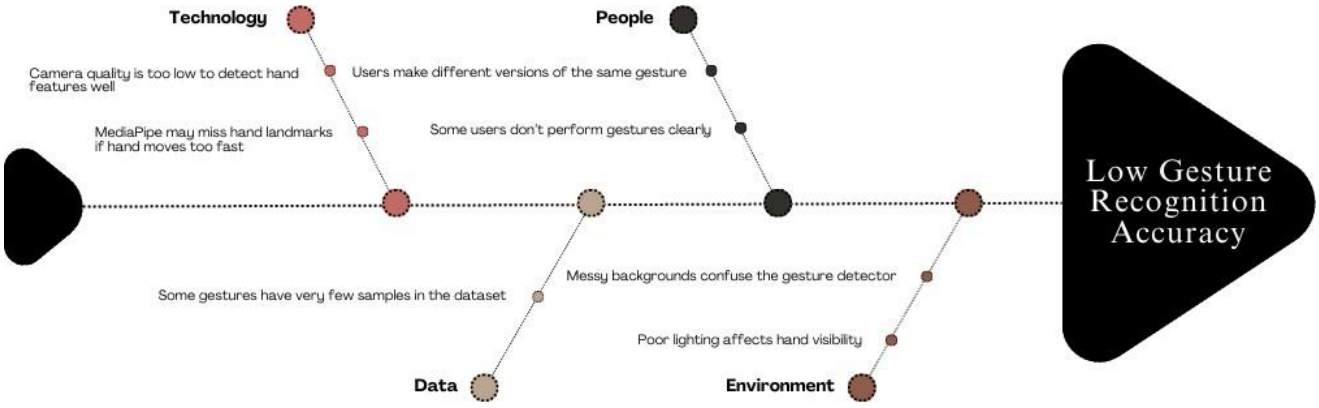


Figure 7: Fishbone Diagram

Functional Decomposition Diagram

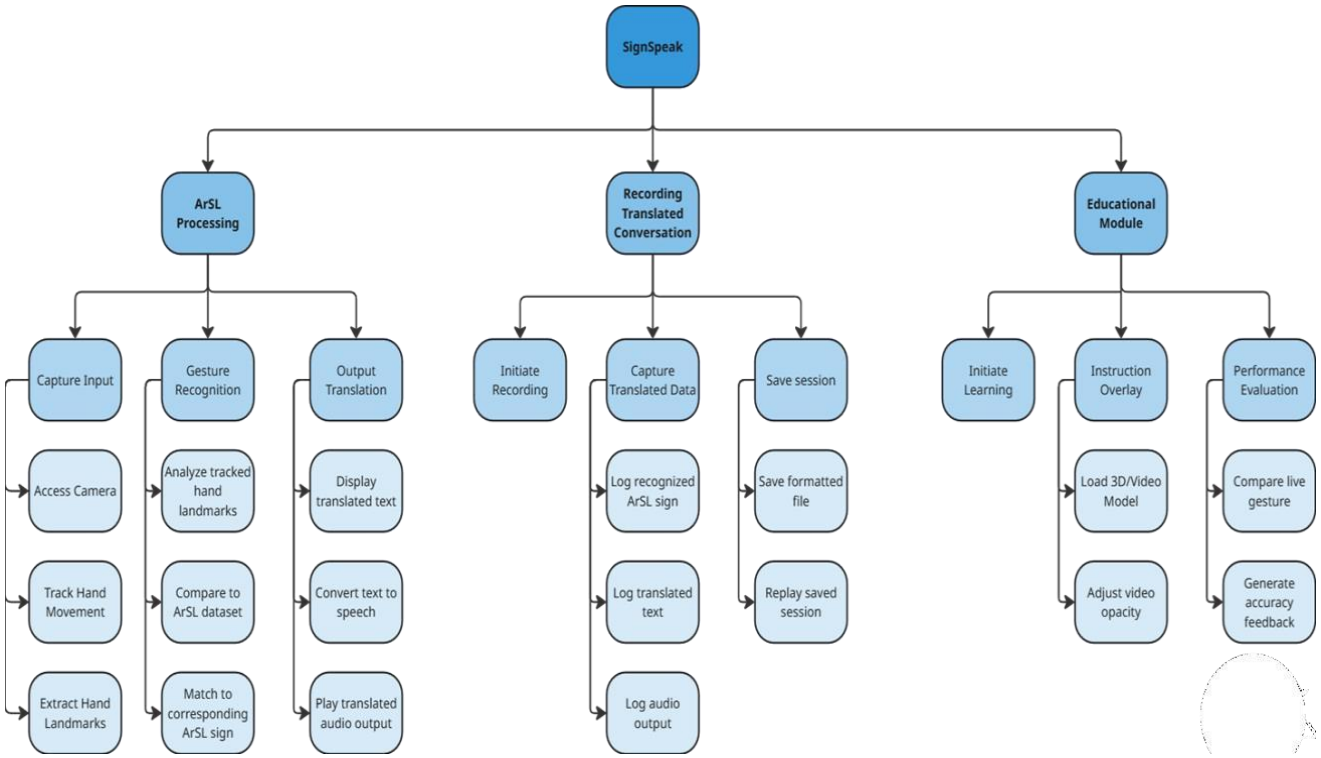


Figure 8: Functional decomposition Diagram

Technical Feasibility

Compatibility checking (hardware / software and other technologies)

Component	Description	Compatibility Check
Software Environment	Programming Language: Python IDE(s): PyCharm and Jupyter Notebook. Libraries and Dependencies: TensorFlow, MediaPipe, OpenCV, Pyttsx3, and NumPy	Compatible with Windows, macOS, Linux, and potentially mobile platforms
Gesture Recognition	MediaPipe is used for real-time performance	Runs efficiently on mid-range CPUs without GPU needed
Webcam Input	Camera to capture real-time hand gestures	Requires standard 720p HD webcam for accuracy
Graphical User Interface	Built for user friendly interaction	Compatible with desktop and potentially mobile platforms
Hardware Requirements	Minimum: Intel i5 / Ryzen 3 8 GB RAM 10 GB storage	Based on the specs of the average individuals' devices
Internet Requirements	Required only for the initial model and dependency download	Supports offline usage after the initial installation
Peripheral Requirements	No external sensors, gloves, or specialized hardware used	Supports accessibility by only requiring webcam

Table 3: Compatibility Checking

Relevance of the Technologies

This project integrates several technologies to enable accurate, real-time Arabic sign language recognition and feedback. OpenCV is used for real-time video capture and image preprocessing, forming the foundation of the input pipeline. MediaPipe is responsible for extracting hand landmarks using the Holistic model, with only the hand landmark component enabled. The system relies exclusively on 21 hand landmark keypoints per hand, as the face and pose landmarks were excluded due to challenges in consistently performing those gestures and to simplify interaction by focusing solely on hand-based signing. The hand landmark model performs keypoint localization for 21 hand-knuckle coordinates within the detected hand regions and is trained on approximately 30,000 real-world images and synthetic hand renderings across varied backgrounds, providing robust and reliable gesture detection. The extracted landmark data is then passed to a feedforward neural network implemented using TensorFlow, which classifies hand gestures. To provide auditory feedback in Arabic, gTTS (Google Text-to-Speech) is used to convert recognized text into speech. Additionally, Arabic-reshaper is employed to ensure the proper formatting and rendering of Arabic script before it is spoken or displayed.

Schedule Feasibility

Gantt Chart:

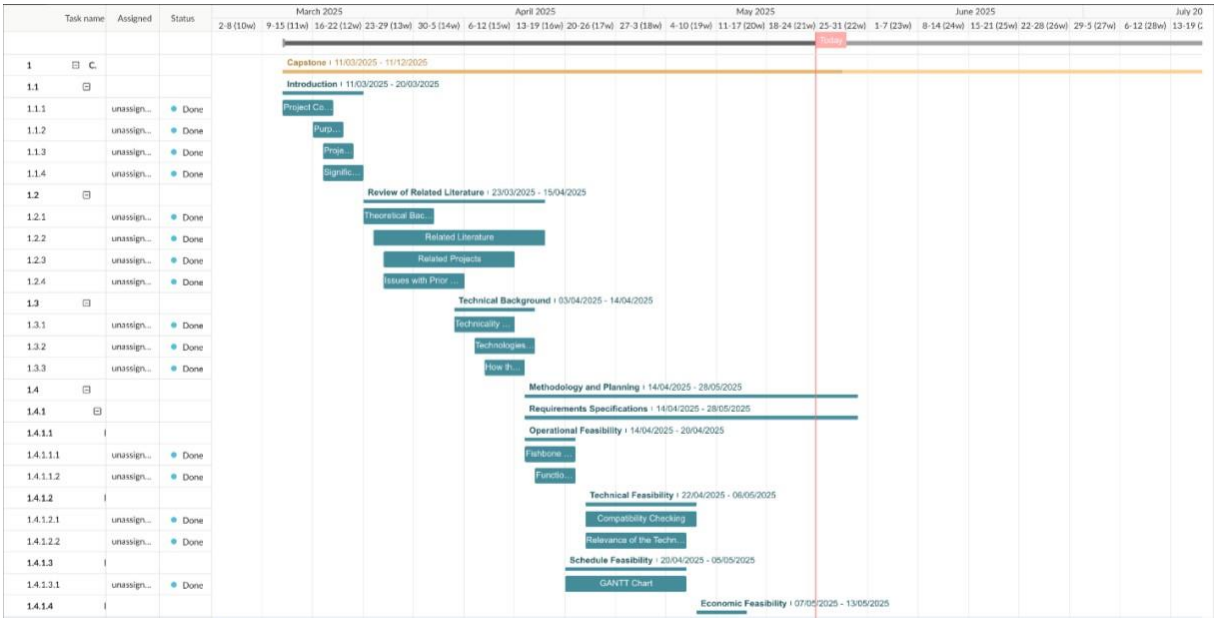


Figure S: Sectioned Gantt Chart

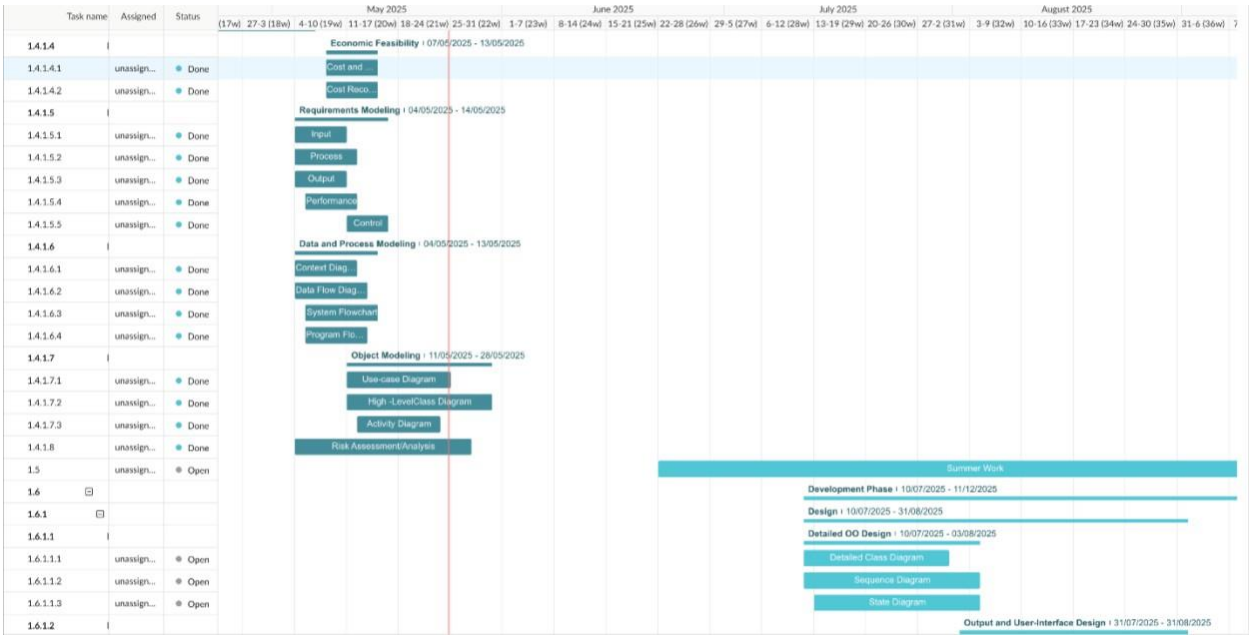


Figure 10: Sectioned Gantt Chart

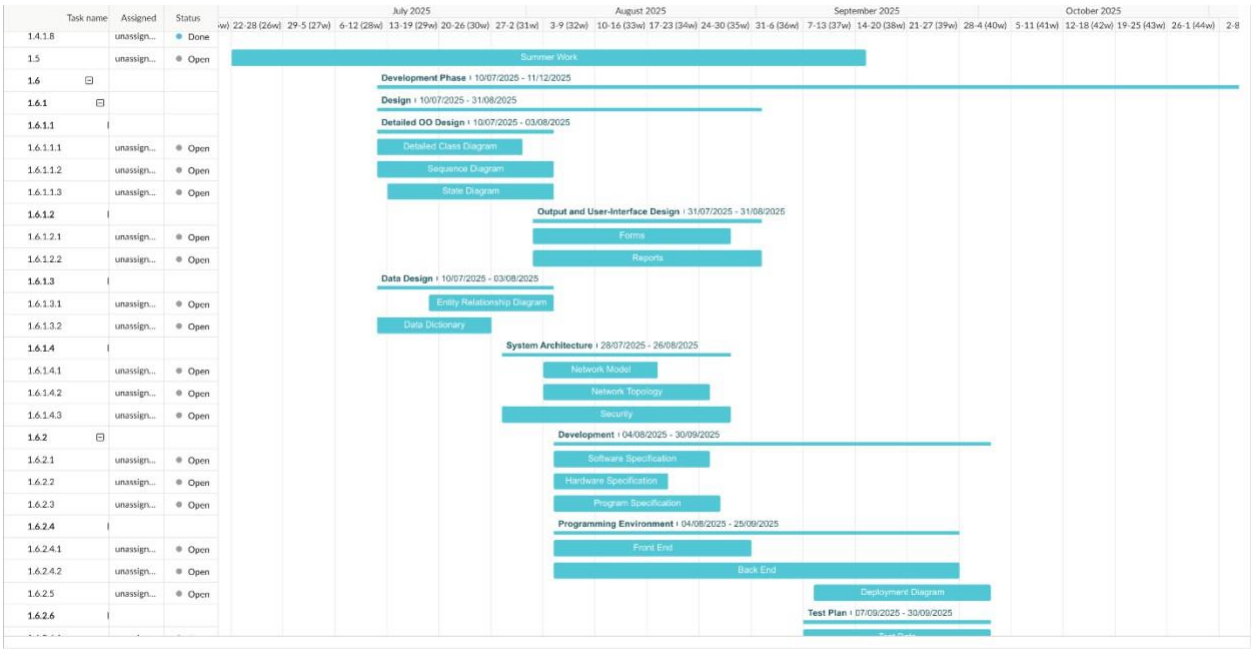


Figure 11: Sectioned Gantt Chart

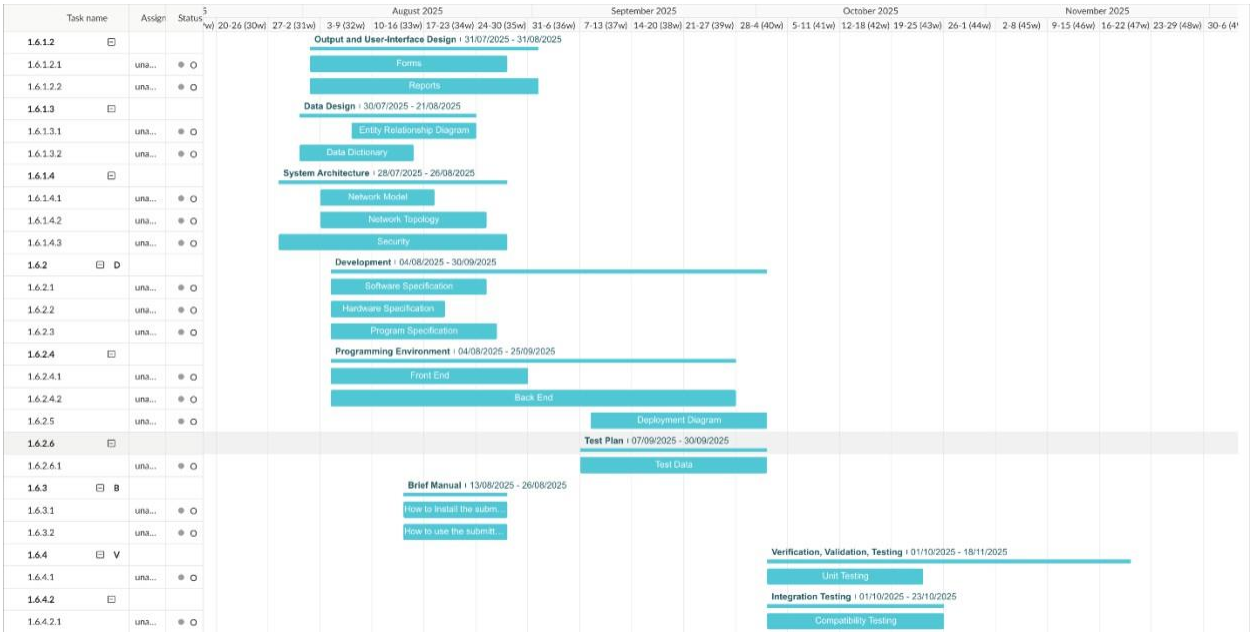


Figure 12: Sectioned Gantt Chart

Figure 13: Sectioned Gantt Chart

Full Gantt Chart:

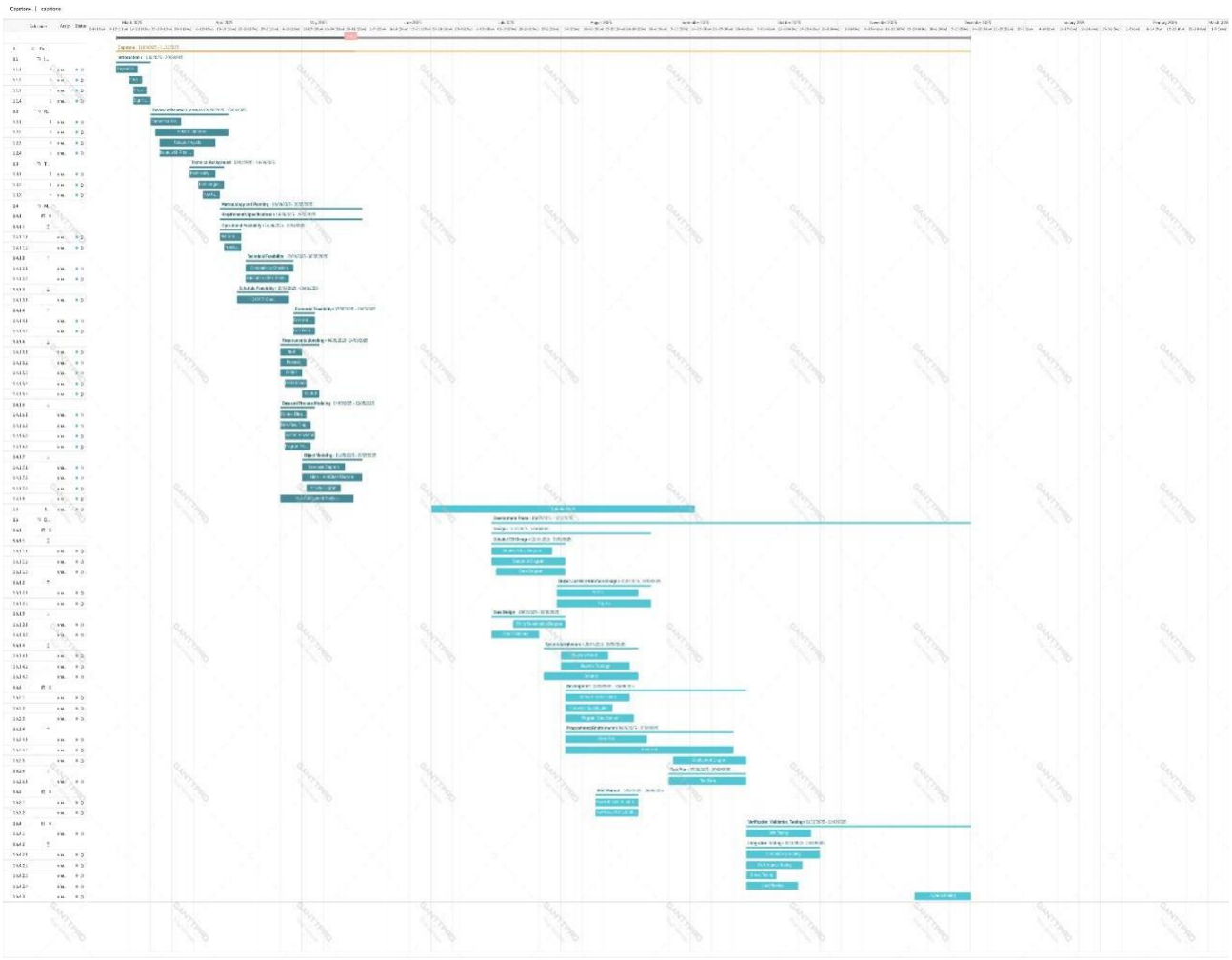


Figure 14: Full Gantt Chart

Economic Feasibility

Economic feasibility consists of three major sections: cost analysis, benefit analysis, and cost recovery scheme. We aim to be transparent and share our resources throughout the project. We have primarily utilized personal hardware and university resources.

Cost and Benefit Analysis

DEVELOPMENT COST (one-time)

Developer Time	Market Rate Equivalence	1000 KWD (4 developers x 250 hrs.)
Hardware (development, testing)	Computers / Laptops / Webcams	500 KWD
Software Licenses	Tools are open source	0 KWD
Dataset Costs	Kaggle is free, assuming a small cost for custom data creation	150 KWD
Internet & Electricity	Utilities, connectivity during development	60 KWD

Table 4: Development Cost

OPERATIONAL COST (recurring)

The value in Kuwaiti Dinars here is what would cost per month.

Hosting (upcoming future online version)	Data storage for educational content, API deployment	15 KWD
Maintenance	Any updates, fixes, or retraining of models	30 KWD
Support	Email / Chat support or documentation maintenance	15 KWD

Table 5: Operational Cost

Benefit Analysis

This evaluates both tangible and intangible benefits.

Benefit	Description	Annual Value Estimate
Reducing the occurrence of depending on an interpreter	Schools, clinics, specialized centers, etc., save money on actual human interpreters	500 KWD per user
Saving Time	Quick interaction in public services, education, etc.	>150 KWD per user
Accessibility Compliance	Organizations can meet accessibility mandates	May vary

Table c: Benefit Analysis

Intangible benefits (ones that cannot be measured) include social inclusion of the deaf and mute community, improved education, and cultural relevance to Kuwait / Arabic-speaking populations.

In the cost recovery scheme, we explain how this investment can be recovered. There are three possible models.

1- A Freemium Model

The basic SignSpeak software, where real-time translation is free but premium features such as session recording and educational modules/advanced analytics are paid. It wouldn't have a high monthly cost, coming in around 2 KD, which is 24 KD annually per user.

2- Institutional Licensing

We would sell a license that is renewed annually to schools, clinics, or government accessibility departments. A sample price would be 90 KD a year per institution.

3- CSR Sponsorship (Corporate Social Responsibility) / Government Grant

This model requires a partnership with the Ministry of Education / Health of Kuwait to subsidize deployment, as this will boost social impact for sponsorship.

Requirements Modeling

Input	• Live webcam • Hand gestures from user (ArSL) • Recorded video
Process	• Capture real-time video input using the device camera. • Extract hand landmarks (21 keypoints) using MediaPipe. • Preprocess landmarks. • Classify the gesture using a trained feedforward neural network (MLP). • Map output class corresponding to Arabic letter/word. • Display recognized text on screen. • Convert text to Arabic speech using a text-to-speech engine.
Output	• Translated ArSL text displayed on screen • Real-time audio output • Saved session files
Performance	• **Our goal is to achieve:** • <1s total response time accuracy > 90%
Control	• Toggle to start/stop of gesture detection • Toggle to start/stop session recording • Selection of input sources (live camera or recorded video) • Navigation controls within GUI (switch between features/pages)

Table 7: Requirements Modeling Table

Context Diagram

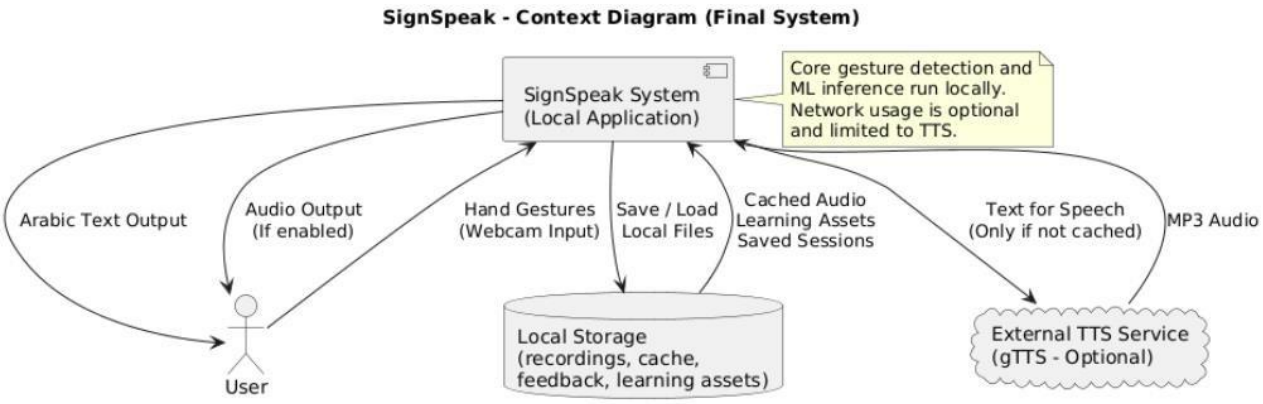


Figure 15: Context Diagram

Data Flow Diagram

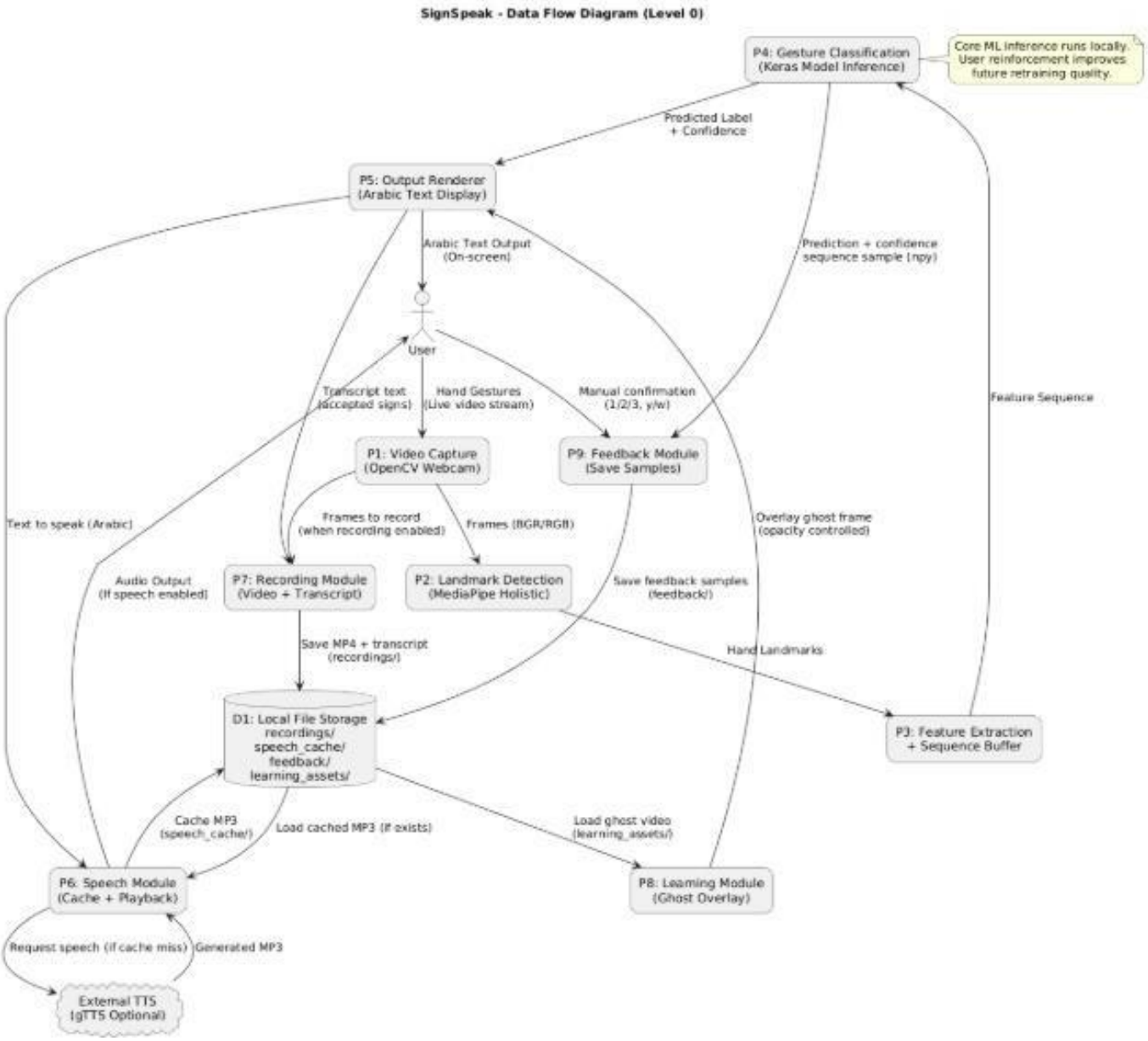


Figure 1c: Data Flow Diagram

Object Modelling

Use Case Diagram

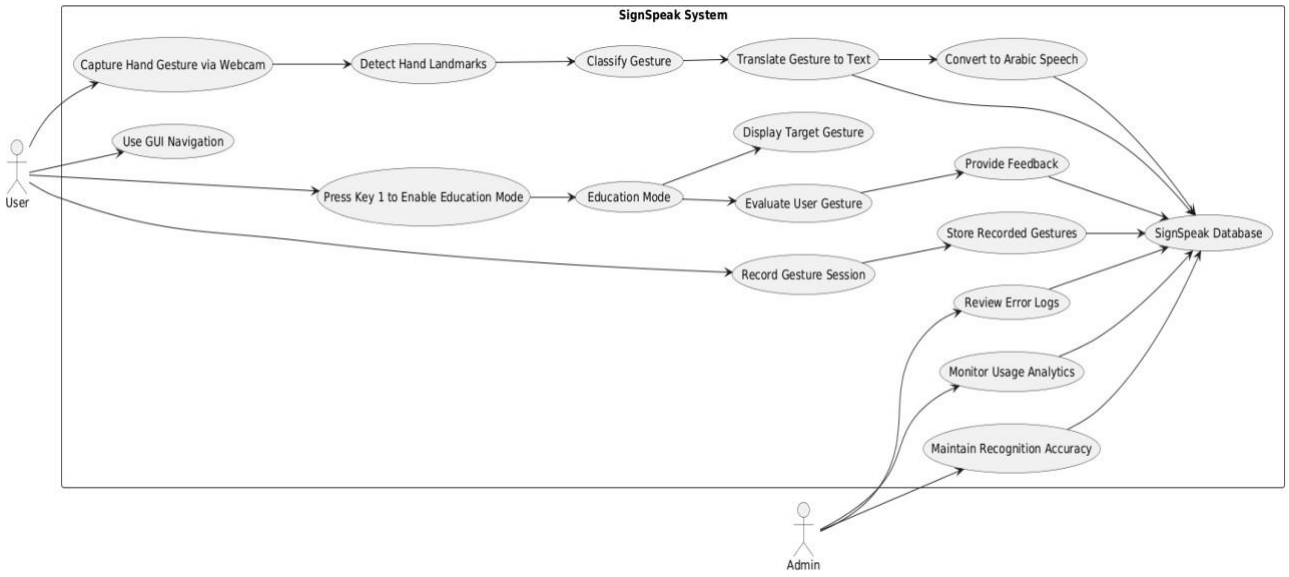


Figure 17: Use Case Diagram

High-Level Class Diagram

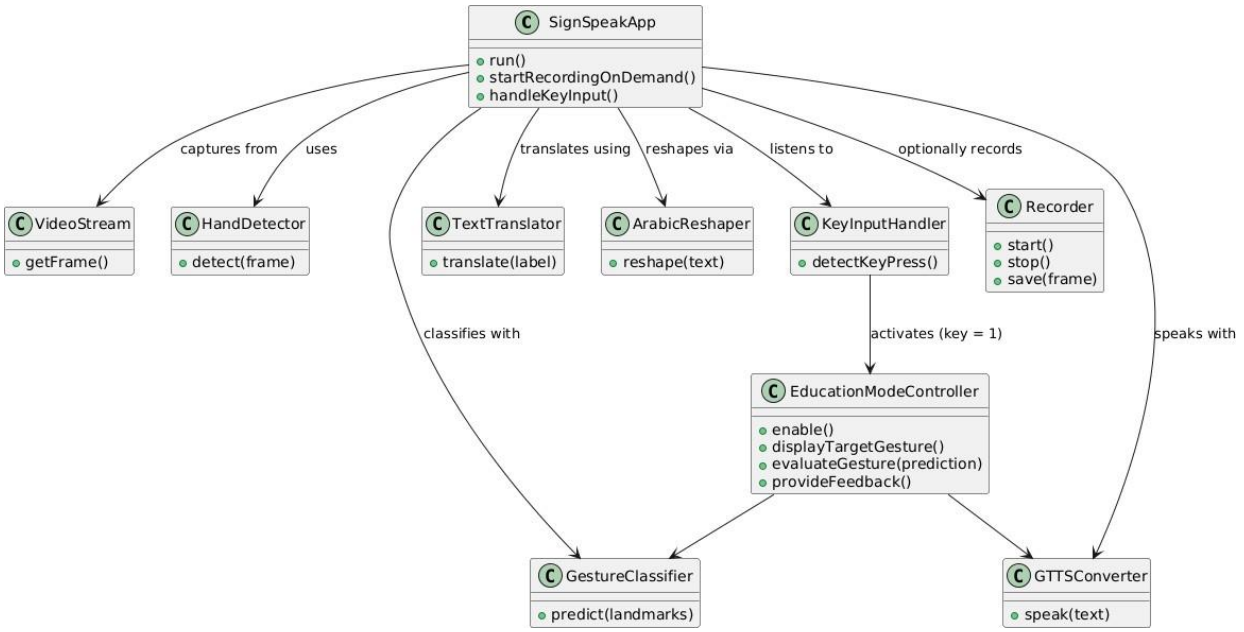


Figure 18: High level class diagram

Activity Diagram

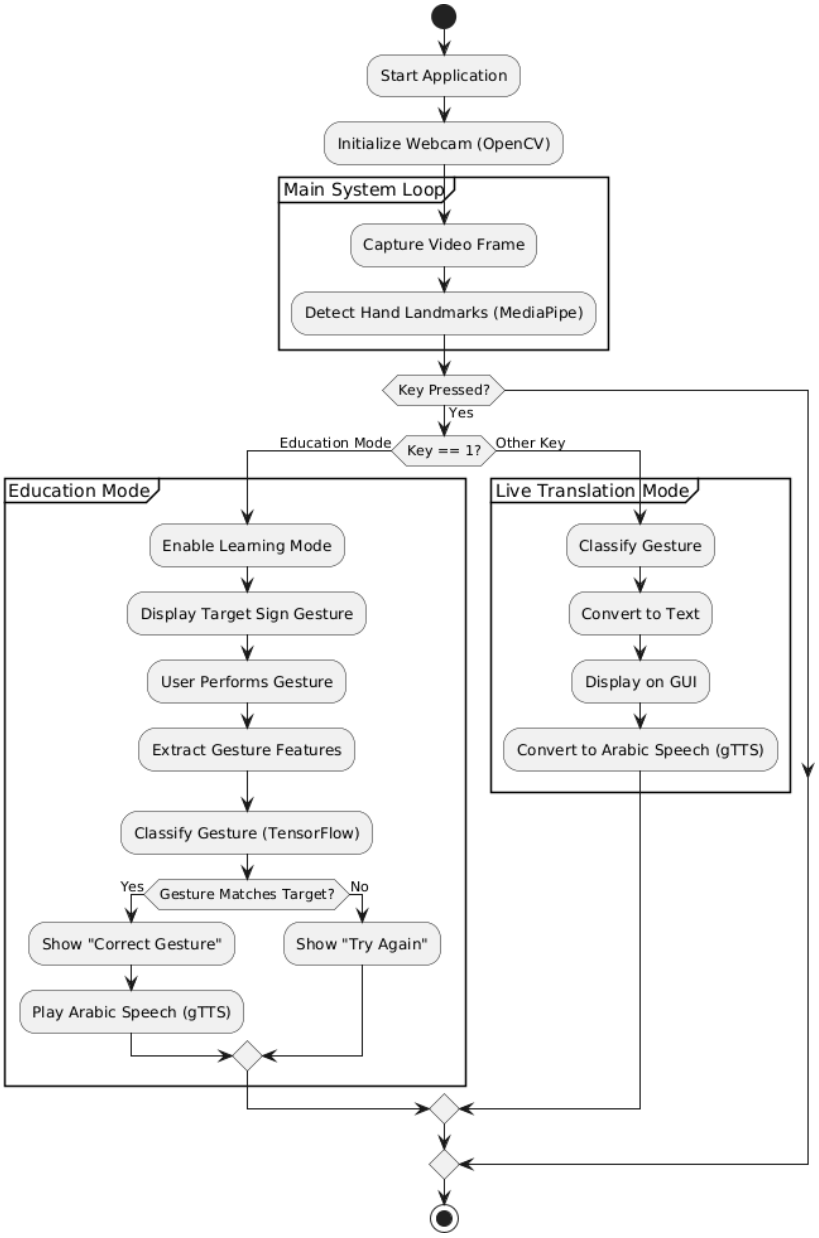


Figure 1S: Activity Diagram

System Flowchart

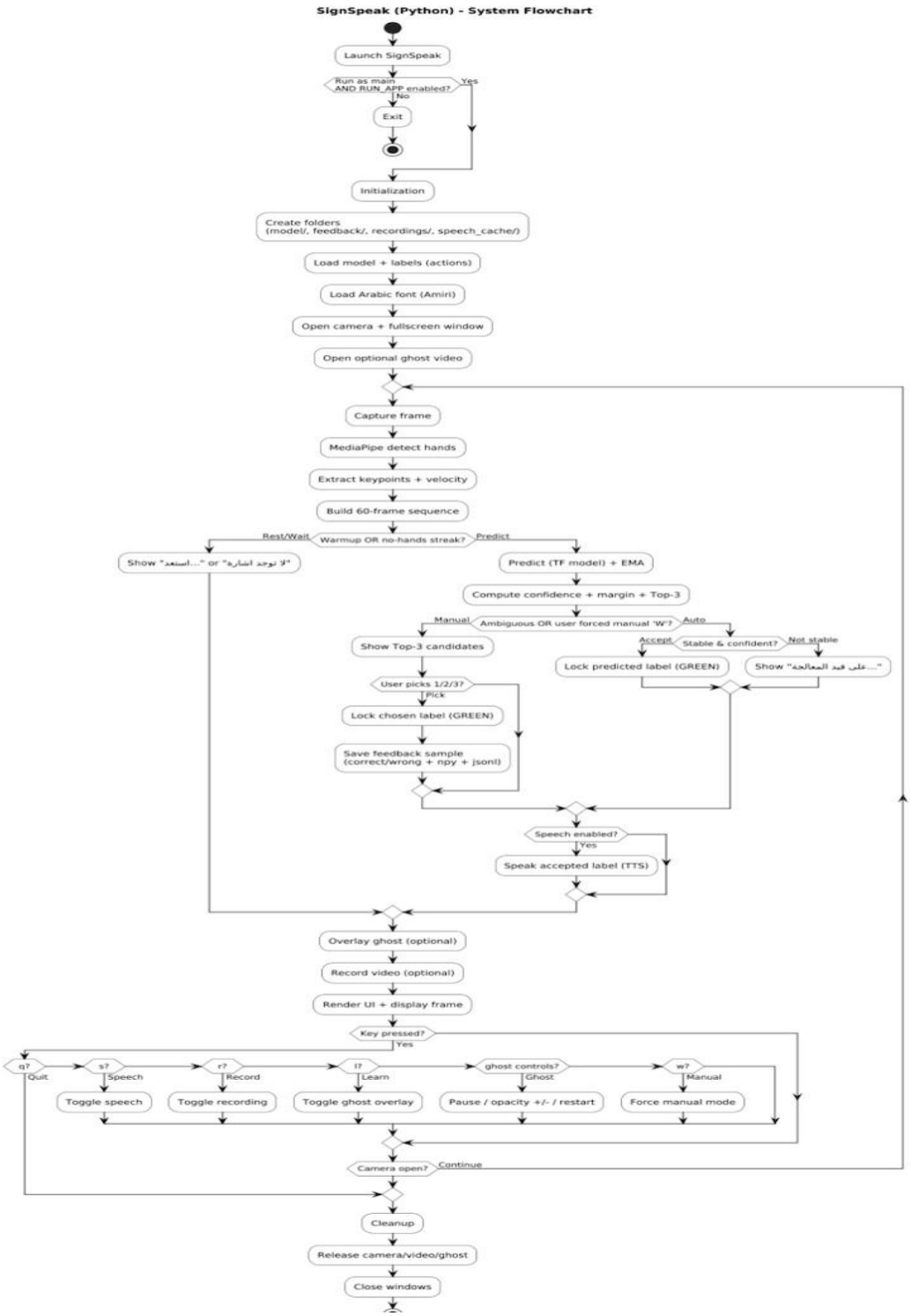


Figure 20: System Flowchart

Program Flowchart

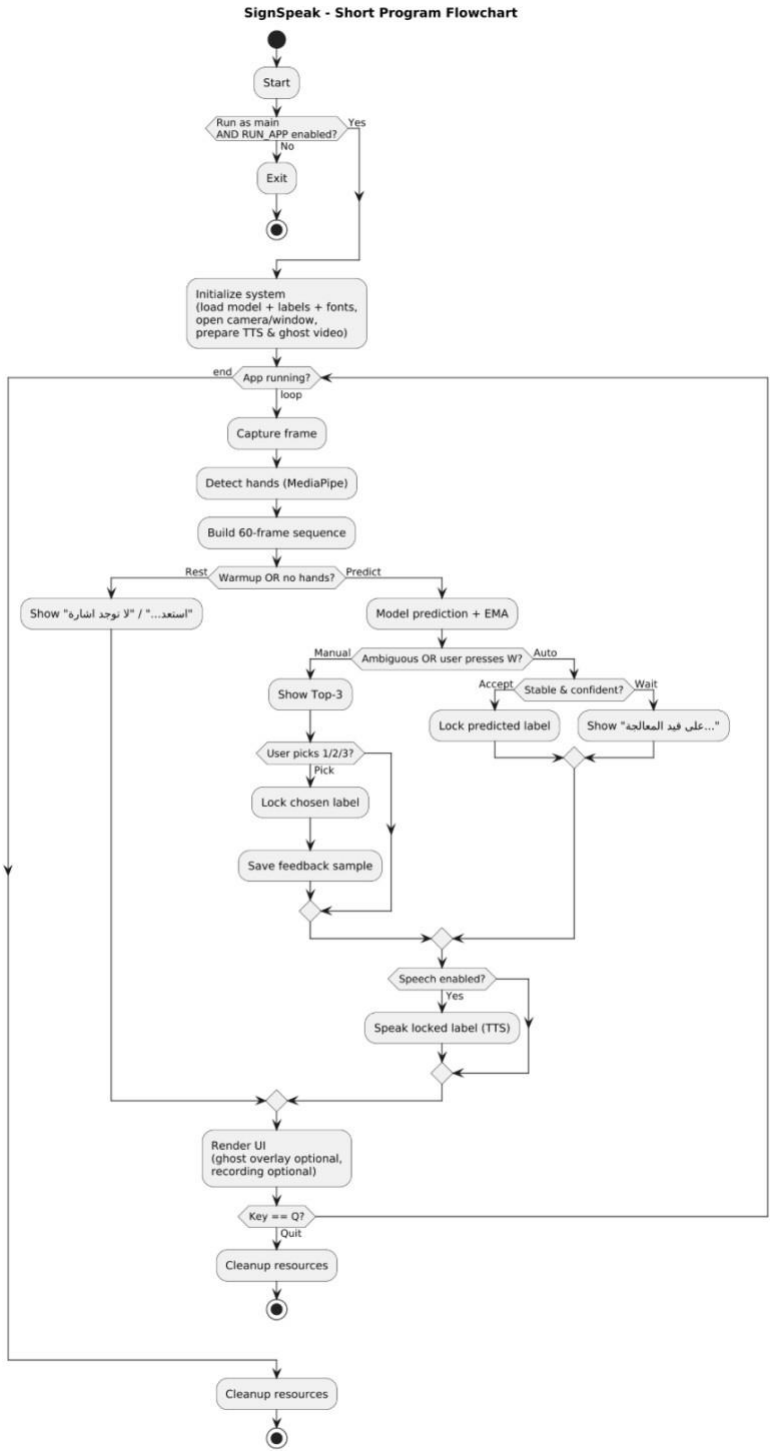


Figure 21: Program Flowchart

Risk Assessment/Analysis

Risk assessment plays a vital role in the development lifecycle of the **SignSpeak** project. It involves identifying, evaluating, and mitigating potential issues that may arise during the software design, implementation, and deployment phases. Given that SignSpeak is a real-time sign language translation system aimed at facilitating communication for deaf, mute, and hard-of-hearing users in Kuwait, it is crucial that the software function with high accuracy, speed, and security.

This section outlines all foreseen risks, categorized by their nature—performance, system, environmental, security, usability, and hardware. Each risk is analyzed based on its root cause, potential response, and its assigned probability and impact rating. This proactive approach ensures that even low-probability issues are considered, helping to deliver a stable and user-friendly application.

The probability/impact matrix is used to visualize the severity of each risk and help prioritize mitigation strategies. Risks marked as both high in probability and high in impact require immediate attention and contingency planning.

Probability/Impact Matrix for Risk Prioritization

	High Impact	**Medium Impact**	**Low Impact**
High Probability	R1	R4	
Medium Probability	R2	R3, R5	
Low Probability		R6	

Table 8: Probability Impact Matrix

Detailed Risk Table

Table S: Detailed risk table

Rank	Risk	Description	Category	Root Cause	Potential Response	Probability	Impact
R1	Gesture misrecognition	Incorrect gesture is translated into wrong text/speech	Performance Risk	Inaccurate gesture-to-text mapping	Improve gesture dataset and re-train the model	High	High
R2	System crash	Software stops responding during use	System Risk	Unhandled exceptions or bugs	Perform extensive debugging and exception handling	Medium	High
R3	Background/environment interference	Gesture not recognized due to poor lighting or clutter	Environmental Risk	Noisy background or insufficient lighting	Use environment-invariant tracking or instruct user guidelines	Medium	Medium
R4	Unauthorized access to recordings	Saved conversation logs accessed by unauthorized users	Security Risk	Lack of proper access control	Implement authentication and encryption of logs	High	Medium
R5	Ambiguity between similar gestures	System confuses two similar hand shapes	Usability Risk	Lack of gesture distinctiveness in dataset	Refine gesture set and train for better classification	Medium	Medium
R6	Hardware limitation (camera)	Poor camera quality leads to inaccurate detection	Hardware Risk	Low-resolution or low-FPS camera	Recommend minimum hardware specs or optimize for low input	Low	Medium

Design

Detailed Class Diagram

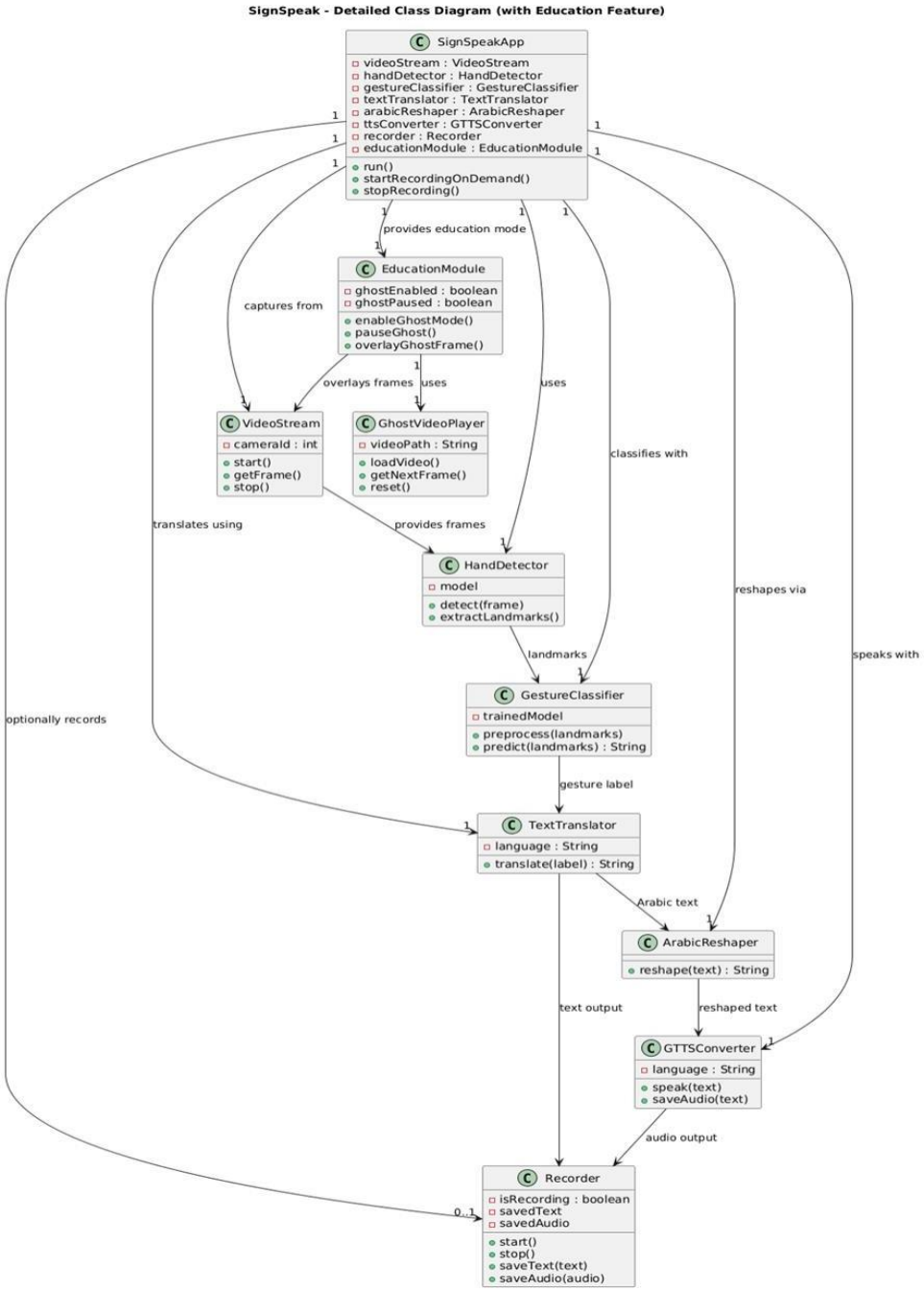


Figure 22: Detailed class Diagram

Sequence Diagram

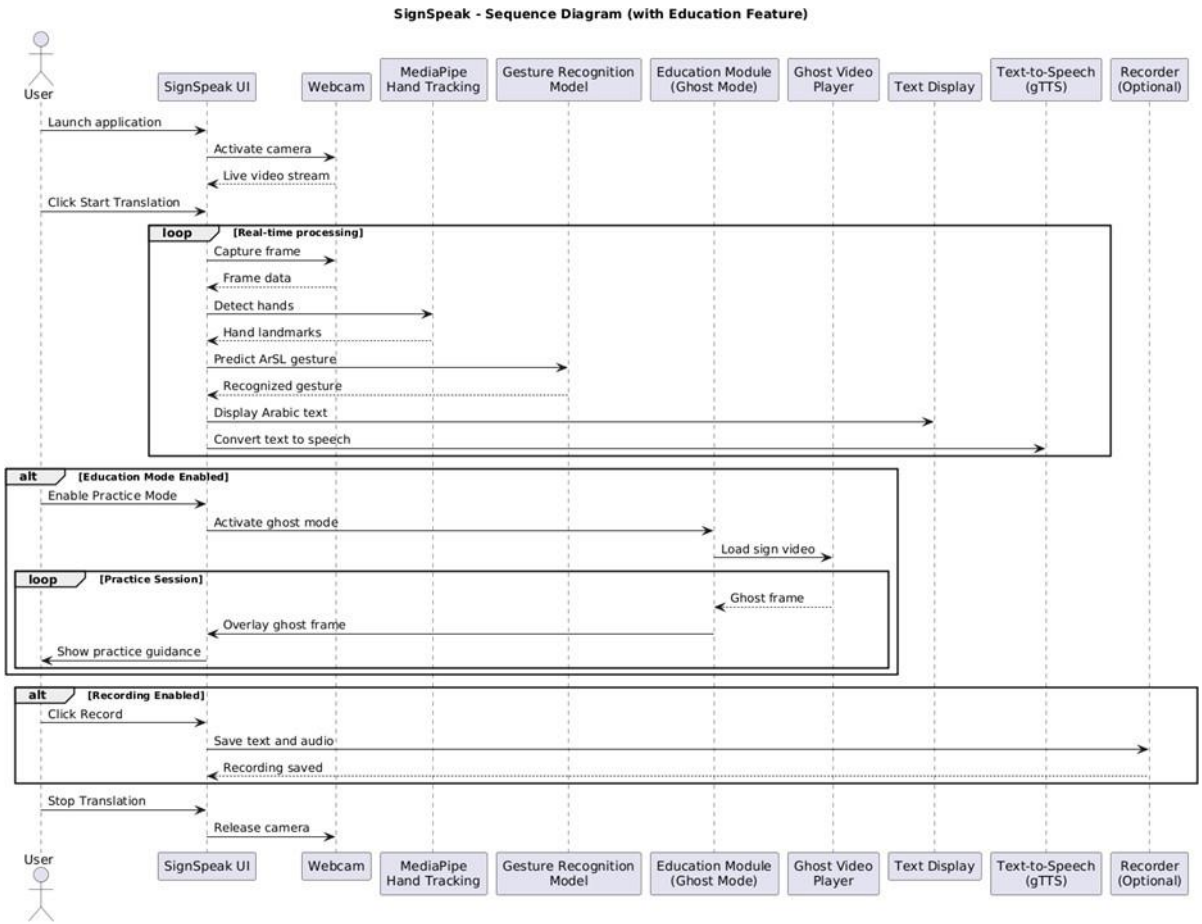


Figure 23: Sequence Diagram

Output and User-Interface Design

Forms

Forms are input data used to help the system user add to, remove, or modify data withing the system. Our proposed sign detection system doesn't have a traditional user interface with interactive system mechanisms (like buttons or drop-down menus). However, our input mechanisms are implemented as logical forms enabling the user to control the behaviors of the system and provide feedback. The following list includes all the forms unutilized in our system:

- **System Control Form:**

  The system includes keyboard-based control inputs to help the user control the flow of the execution. Pressing on" q" key terminates the application safely, ensuring a controlled shutdown of the system.

- **Prediction Confirmation Form:**

Relying completely on the model to make predictions of the signs can result in inaccurate outputs, especially when a given sign is visually similar to another sign in the system. To address this issue, we implemented a user reinforcement mechanism where the user can reinforce the prediction by selecting one of the three most probable signs. The user is prompted to choose between 1, 2, or 3 (since the system currently supports three signs), and the selected sign is then displayed in green to provide clear visual feedback. Furthermore, the chosen prediction is saved in a dedicated file for correct predictions, which can be used later to fine-tune the model and improve future accuracy.

- **Speech Output Activation Form:**

The user can activate the speech function by pressing on the "s" key. This form provides auditory feedback by converting text to speech.

- **Recording Form:**

The users can record their sessions by pressing on the "r" key. The sessions will be saved as an mp4 file along with the speech if it was enabled during the recording.

- **Learning Form:**

By pressing the l key, the user is presented with a low-opacity video representing the 3D model for a given sign. This form allows the user to learn and practice the sign while simultaneously viewing their own gestures, and the system is able to predict whether the performed gesture is correct or incorrect.

Reports

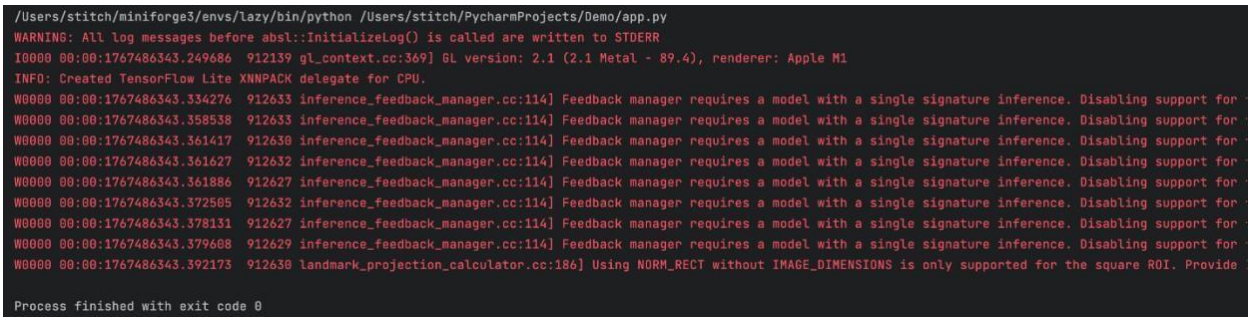


Figure 24: terminal output after terminating our program

```
print("Train samples:", X_train.shape[0], "Test samples:", X_test.shape[0])
print("Unique train labels:", len(np.unique(np.argmax(y_train, axis=1))))
print("Unique test labels:", len(np.unique(np.argmax(y_test, axis=1))))
```

```
Train samples: 1680 Test samples: 420
Unique train labels: 35
Unique test labels: 35
```

Figure 25: number of train, test samples, and unique labels in the model

```
accuracy_score(ytrue, yhat)
```

[32]:

0.9142857142857143

Figure 31: achieved accuracy score

Data Design

Entity Relationship Diagram

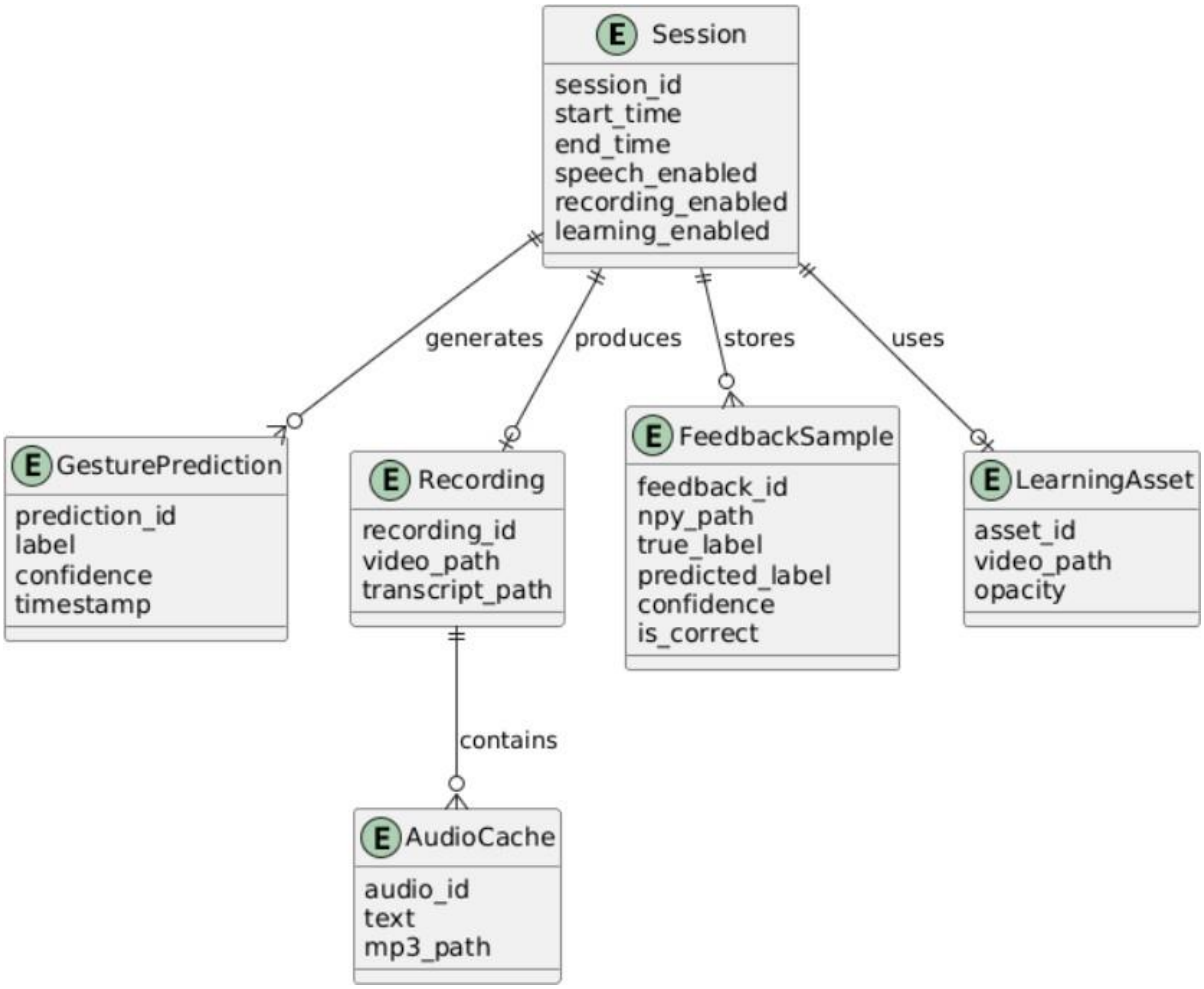


Figure 32: Entity Relationship Diagram

The Entity Relationship Diagram (ERD) demonstrates a conceptual data model rather than a physical database schema because SignSpeak does not rely on a conventional database system. The logical relations between runtime sessions, gesture predictions, recordings, feedback samples, and learning assets produced during system execution are shown in the ERD. These entities are files and in-memory structures created by the application, including feedback datasets for model improvement, cached speech files, and recorded videos.

Data Dictionary

Models and Labels

Table 10: Models and Labels

Variable	Type	Description
model	TensorFlow Keras Model	Loaded from model.keras or model.h5.
actions	NumPy Array (Strings)	A list of 35 Arabic signs (e.g., "السلام عليكم", "شكرًا"). Index 0 is "No Sign".
Sequence	List	A sliding window of the last 60 frames of hand movements used for prediction.

Hand Key Points

Table 11: Hand Key points

Feature	Size	Description
pos_126	126 values	The (x, y, z) coordinates for 21 points on the left hand and 21 on the right hand.
Vel_126	126 values	The velocity (speed and direction) of the hand points (current position minus previous position).

Feat_252	252 values	The final combined data for one frame: pos_126 + vel_126.

UI Colors

Table 12: UI Colors

Color (BGR)	Meaning
Blue (255, 0, 0)	Rest: No hands detected; looking for a sign.
Red (0, 0, 255)	Processing: Hand movement detected; calculating.
Green (0, 255, 0)	Accepted: A sign has been successfully recognized.

Logic

Table 13: Logic

Variable	Type	Description
no_hands_streak	Integer	Counts how many frames have passed without seeing a hand. Used to reset the app.
pred_history	Deque (List)	Stores the last few predictions. A sign is only locked in if it is consistent (stable).
manual_mode	Boolean	True if the AI is unsure and asks the user to pick between top choices (1, 2, or 3).
ema_res	NumPy Array	Exponential Moving Average. It smooths out the AI's confidence scores so the results don't flicker.

Feedback and Training Data

Table 14: Logic

Field	Description
seq60x252	A file containing the 60 frames of movement (saved as a .npy file).
true_label	The sign the user actually picked.
proposed_label	The sign the AI thought it saw.
confidence	How sure the AI was (0.0 to 1.0).
ok	A "Yes/No" flag. True if the AI guessed correctly, False if the user had to correct it.

Thresholds

Table 15: Thresholds

Variable	Value	What it means
ACCEPT_CONF	0.95	The AI must be at least 95% sure to automatically accept a sign.
STABLE_K	5	The AI must see the same sign for 5 frames in a row before it's stable.
COOLDOWN_SECONDS	1.0	The minimum wait time between two different signs.
AMBIG_MARGIN_MAX	0.12	If the gap between the #1 and #2 guess is smaller than this, the AI asks for help (Manual Mode).

Audio and Speech Data

Table 1c: Audio and Speech Data

Item	Description
speech_cache/	A folder where the app saves .mp3 files of the words so it doesn't have to download them every time.
safe_name	The text of the sign (e.g., "شكراً") converted into a filename (e.g., شكراً.mp3).
lock	A safety key that prevents the app from trying to play two sounds at the exact same time.

Ghost Video (Educational Layer)

Table 17: Ghost Video (Educational Layer)

Variable	Description
ghost_cap	The video file being read from the education/sign.mp4 path.
gh_alpha	The opacity level (how see-through it is), controlled by the slider (0.0 to 1.0).
ghost_paused	A True/False switch to freeze the teacher's video while you practice.

System Architecture

Network Model

SignSpeak uses a network model for standalone local execution. The system's fundamental operations are independent of any backend servers or constant network connectivity. Using a privately deployed model, all computer vision processing, gesture recognition, and machine learning inference are carried out locally on the user's device.

When the user enables it, an external text-to-speech service is used to produce Arabic speech output. To reduce recurrent network usage, generated audio files are cached locally. With the exception of this optional feature, no user gesture data, video frames, or private information are sent over the network when the system is in offline mode.

Network Topology

The SignSpeak system has a single-node network topology. The webcam, processing unit (CPU/GPU), storage, and machine learning model are all part of the same local machine. Distributed processing or inter-node communication are not involved.

By removing the need for distant servers or external data transmission during gesture recognition and translation, this streamlined topology improves system reliability, lowers latency, and supports user privacy.

Security

The SignSpeak system is built around a robust, privacy-first, offline architecture that ensures maximum data security by minimizing potential attack surfaces. All video processing occurs entirely on-device: video frames are captured, and all subsequent steps, including MediaPipe hand-landmark extraction and the model's prediction pipeline, are processed locally without being uploaded, transmitted, or logged. This design eliminates risks associated with network interception, cloud data leaks, or unauthorized remote server access.

To further bolster security, the system has no personal profiles, no authentication system, and no dependence on databases or cloud APIs, meaning there is no sensitive data to compromise. Secure spoken outputs are also locally generated without relying on external APIs or Google servers, ensuring that audio data never leaves the system. Access to hardware is strictly

controlled: the only required permission is for the webcam, and camera access occurs strictly within OpenCV, with no external libraries accessing the feed and no image saving permitted.

Finally, all external model files are loaded from the local file system. In essence, real-time camera data never leaves the device, and all AI functions—including text-to-speech (TTS)—operate locally, making the system inherently secure, transparent to users, and ideal for deployment in highly confidential environments like schools, hospitals, and government institutions.

Development

Software Specifications

Table 18: Software Specifications

Component	Specification	Usage
Programming Language	Python 3.9 - 3.11	The programming language used to implement the software components
Development Environment	• Pycharm • Jupyter Notebook	The IDE's used to develop, debug, and test the software
Libraries & Frameworks	• MediaPipe **0.10.21** • OpenCV 4**.11.0.86** • TensorFlow **2.19** • Keras **3.10** • Scikit-learn **1.6.0** • NumPy **1.26.4** • playsound3 **3.3.0** • Matplotli **3.10.7** • arabic-reshaper **3.0.0** • Pillow **10.0.0** • gTTS **2.5.4**	The libraries & frameworks used to provide hand landmark detection, real-time video processing, deep learning inference, and numerical computation
Operating Systems	• Windows • MacOS	The supported platforms for the software development and deployment

GUI & Output	• Google-Text-to-Speech (gTTS) • opencv • Arabic-Reshaper	The Opecv camera, the text, speech, and the different actions the user can perform by pressing on certain keys are considered a part of the GUI.
Datasets	custom-collected gestures dataset	Used for training and improving the gesture recognition model
Internet Requirement	Only needed during initial setup	Used to download dependencies, libraries, and datasets

Hardware Specifications

Table 1S: Hardware Specifications

System Type	Component	Specification	Purpose / Usage
Gaming PC (Development System)	CPU	Intel Core i7-12700H (12-Core, up to 4.7 GHz)	Used for software development, debugging, and real-time testing.
	RAM	16 GB DDR4 (3200 MHz)	Ensures smooth execution of MediaPipe, TensorFlow, and GUI simultaneously.
	Storage	512 GB NVMe SSD	Fast data access for model files, datasets, and recorded sessions.
	GPU	NVIDIA RTX 3050 (4 GB VRAM)	Accelerates training and improves inference speed during testing.
	Webcam	Logitech C920 HD Pro (1080p, 30 fps)	Provides clear and stable image input for accurate hand-tracking.

	Display	15.6” FHD (1920×1080)	Used for visualizing output text and GUI elements.
	Operating System	Windows 11 (64-bit)	Primary development environment for Python, TensorFlow, and OpenCV.
University High-Performance PC (Training System)	CPU	Intel Core i9-13900K (24-Core, 5.8 GHz)	Used for large-scale dataset preprocessing and deep model training.
	RAM	32 GB DDR5	Handles intensive TensorFlow operations and multitasking.
	Storage	1 TB SSD	Stores large training datasets, checkpoints, and model outputs.
	GPU	NVIDIA RTX 3080 (10 GB VRAM)	Enables faster model training and higher-accuracy inference.
	Webcam	Logitech C922 Pro Stream (1080p, 60 fps)	Used during high-quality model validation sessions.
	Operating System	Ubuntu 22.04 LTS (64-bit)	Optimized for TensorFlow GPU acceleration and training stability.
	Internet Connection	Stable 10 Mbps or higher	For downloading datasets and synchronizing models.

Program Specifications

SignSpeak is a real-time Arabic Sign Language translation tool that uses a live camera to recognize hand gestures, turning them into easy-to-read Arabic text and clear speech. Instead of a typical visual interface, the system is controlled using simple keyboard commands.

The app works by capturing video from a camera, identifying hand positions with MediaPipe, and analyzing movement patterns to understand each gesture. It then uses an advanced deep learning model to figure out what sign is being shown. Once a sign is recognized, it appears on the screen in Arabic and can also be spoken aloud if needed.

The system incorporates session recording, learning assistance via a semi-transparent instructional hand model, and a user feedback mechanism that enables manual correction of ambiguous predictions. It operates locally and does not require continuous internet connectivity.

Programming Environment

Front End

N/A

Back End

N/A

Deployment Diagram

Test Plan

The SignSpeak system test plan was created to make sure the system is correct, stable, easy to use, and reliable before it is fully launched. We tested each part step by step, beginning with single components and moving up to the whole system.

The stages of the testing procedure were as follows:

1- Arabic text rendering, feature extraction, model inference, hand landmark detection, and text-to-speech generation are examples of core modules that undergo unit testing.

2- Testing for compatibility with various hardware setups and operating systems.

3- System accuracy, responsiveness, and real-time behavior are measured through performance testing.

4- To assess system stability under extended use and challenging circumstances, stress and load testing are used.

5- To confirm that all functional and non-functional requirements were fulfilled, conduct final system testing.

Before going on to the next testing phase, all problems found during testing were fixed, guaranteeing a stable and dependable final system.

Test Data

SignSpeak underwent evaluation using real-time webcam input of Arabic Sign Language gestures performed by different users. The test dataset encompassed static and dynamic gestures across various conditions, including lighting, background complexity, camera distance, and gesture speed. Furthermore, additional test data comprised recorded video sessions, cached text-to-speech audio files, and feedback samples collected during user confirmation and learning modes. This data proved sufficient to assess system accuracy, stability, and behavior under normal and stressed usage conditions.

Brief Manual

How to install the submitted software:

- Installation Steps

1. Install Python 3.x (version 3.9–3.11 recommended).
2. Install required Python packages individually:

   pip install mediapipe==0.10.21

   pip install opencv-python==4.11.0.86

   pip install tensorflow==2.19

   keras==3.10

   scikit-learn==1.6.0

   pip install numpy== 1.26.4

   pip install playsound3== 3.3.0

   pip install matplotlib== 3.10.7

   pip install arabic-reshaper==3.0.0

   pip install gTTS== 2.5.4

   pip install pillow

3. Ensure your webcam is connected and working.
4. Download or copy the SignSpeak project folder to your computer.

- Running SignSpeak

1. Open a terminal or command prompt and navigate to the SignSpeak folder.
2. Run the software using:

python app.py

3. When the camera window appears, position your hands within the visible frame.
4. Perform Arabic Sign Language gestures.
5. The software will:
    - Show recognized gestures as Arabic text
    - Generate speech output in Arabic

How to use the submitted software:

- Using the Software
    - Start / Stop: Control real-time gesture recognition.
    - Record: Save your session, including gestures, text, and audio.
    - Playback: Review saved sessions using the GUI or stored files.
    - Tips for Accuracy:
        - Use good lighting
        - Keep hands clearly visible
        - Avoid cluttered backgrounds
- Troubleshooting

Table 20: Issues and solutions

Issue	Solution
Camera not detected	Check connection and permissions
Gesture not recognized	Adjust lighting, slow hand movements
No audio output	Check gTTS installation and ensure internet was available during setup
Missing library errors	Install the missing package individually with pip install package-name

Verification, Validation, Testing

Unit Testing

Unit testing was conducted to verify the correctness and stability of the individual components of the SignSpeak system. Each core module was tested independently to ensure proper functionality before system integration.

The unit tests covered hand landmark detection, feature extraction, trained model loading and inference, Arabic text rendering, text-to-speech generation, feedback data storage, and runtime control operations. All tests were executed successfully, and the system ran without runtime errors, crashes, or unexpected behavior.

Table 21: Unit testing

Test Case	Module Tested	Description	Result
UT-1	Model Loading Test	Verified trained model loads correctly	Pass
UT-2	Feature Extraction Test	Verified correct generation of hand landmark features	Pass
UT-3	Prediction Test	Verified model inference executes without errors	Pass
UT-4	Arabic Text Rendering Test	Verified Arabic text displays correctly on screen	Pass
UT-5	Text-to-Speech Test	Verified Arabic speech is generated and played	Pass
UT-6	Feedback Storage Test	Verified prediction data is saved successfully	Pass
UT-7	Runtime Control Test	Verified user controls function correctly	Pass

Integration Testing

Integration testing is implied implicitly in the system testing. Please refer to page 77-78 where you can find system testing.

Compatibility Testing

Compatibility testing was conducted to ensure that the SignSpeak system operates correctly across different operating systems and hardware configurations. The system was tested on multiple platforms and demonstrated consistent behavior and stable performance.

SignSpeak runs successfully on major operating systems, including Windows, Linux, and macOS, provided that the required dependencies are installed. No platform-specific issues were observed during testing, confirming that the system is platform-independent and portable.

Performance Testing

The program launches cleanly and the detection performs well, achieving 91% accuracy. The text displays perfectly, and the speech function works immediately when pressing “s” and stops when pressed again. The recording and learning functions also work well when pressing “r” for recording and “l” for learning and pressing them again stops the process. The program terminates cleanly when pressing “q”. There are no crashes in the system. The detection is not always correct, and in some cases the model may produce an incorrect prediction with a very high confidence (e.g., 95%).

Stress Testing

We performed stress testing to see how the SignSpeak system works in tough conditions that go beyond normal use. This helped us find the system’s limits and see how stress affects its accuracy and stability.

We tested the system in different environments, such as low or uneven lighting, busy backgrounds, greater distance from the camera, and fast or exaggerated hand movements. We also tried repeating gestures for a long time, switching often between system modes, and making gestures partly outside the camera's view.

The results showed that while the system remained stable and did not crash or freeze, recognition accuracy decreased under poor lighting conditions, excessive background noise, fast hand motion, and improper hand positioning. These factors occasionally caused incorrect predictions or delayed recognition. However, the system recovered immediately once normal conditions were restored, and no permanent degradation in performance was observed.

Overall, the stress testing confirmed that SignSpeak is robust in terms of stability, but like most computer vision–based systems, its accuracy is sensitive to environmental conditions such as lighting quality, camera distance, and motion speed.

Load Testing

The system was tested continuously for more than half an hour by performing gestures, turning the audio on and off, and toggling the voice feature on and off. The system worked smoothly throughout the testing period, with no crashes, freezes, or noticeable performance degradation. The sign detection process wasn't affected by how much time the program was running for. It didn't get better or worse; the detections remained the same. However, the system requires a high-resolution camera and at least 8 GB of RAM to perform smoothly.

System Testing

All the functions of the system work together smoothly and without any crashes or suspicious behaviors. We will illustrate the testing for our system functions using the table:

Table 22: System testing

Functionality	Testing
Turning on the system	The system will be turned on by running the programming, which is executing smoothly and consistently without crashes.
Sign detection	This function detects the signs performed by the user and presents the three most probable predictions. The user is prompted to choose between the three signs by pressing 1/2/3, allowing the system to save the correct feedback and reuse it for fine-tuning the model. If the user does not select any of the three signs, the model automatically presents the sign with the highest prediction in green. This function works as intended.
Display Text	This function displays the Arabic text on the screen. It also works as intended.
Turning off the system	The system will be turned off by pressing on "q" key. The termination process is working as intended.
Turning on the audio	The audio will be turned on when the user presses on "s" and off if we press again. This function is also working perfectly.
Turning on the learning	The user will be prompted with a low-opacity video of a 3D model (initially). The user can take control of adjusting the opacity of the video. The sign detection works side by side with the learning feature, allowing the user to see themselves while gesturing, while the sign detector remains active to determine whether the sign was performed correctly. This feature works smoothly and is activated by pressing

	“l” and deactivated by pressing it again.

Conclusions

A real-time Arabic Sign Language recognition and translation system that seamlessly combines computer vision, machine learning, and assistive technology is successfully delivered by the SignSpeak project. Using a webcam, the system can identify hand gestures, extract useful features, classify signs accurately, and display the results in Arabic text, speech output, recording, and an interactive learning mode.

Usability, dependability, and privacy were prioritized throughout the development process. With a privately deployed machine learning model, the system runs mostly offline, guaranteeing low latency and safeguarding user data. Without sacrificing system stability, optional features like text-to-speech and session recording improve accessibility and user engagement. Furthermore, by incorporating user feedback and reinforcement mechanisms, the system is able to gather important information for future model enhancement.

Thorough testing has also proved that SignSpeak works efficiently in a real-world setting and functions in a stable manner when used for an extended period. Although certain factors, such as lighting, complexity, and speed of movement, can impact the recognition performance, the system still works properly. Moreover, the successful implementation of the learning feature further adds to the system’s capabilities, which not only translates but also acts as a source for communications and a means for learning.

In summary, SignSpeak achieves its purpose and has a good foundation for any possible improvements, such as sign vocabulary, resistance of errors, and portability of the system on any mobile device.

Recommendations

Looking ahead, several enhancements can significantly expand the capabilities and impact of SignSpeak. One important recommendation is the development of a mobile application version to increase accessibility for everyday use, particularly for users who rely primarily on smartphones. In addition, expanding the gesture dataset to include more dialects and regional variations of Arabic Sign Language across the MENA region would greatly improve accuracy and inclusivity.

Integrating more advanced deep learning models, such as Transformer-based vision models or lightweight, mobile-optimized neural networks, could further enhance recognition speed and accuracy while reducing computational load. Future versions of SignSpeak could also benefit from two-way translation, enabling not only sign-to-speech but also speech-to-sign animations through the use of 3D avatars. For educational purposes, the planned practice mode could evolve into a full learning platform that offers personalized feedback, interactive exercises, and progress tracking to support users in improving their signing skills.

We also aim to design an AI agent that leverages detection feedback to fine-tune predictions with assistance from a developer on the other end. After detecting a sign, the prediction would be passed to the agent to verify and confirm its accuracy. Implementing this feature would require the collection of user data, for which explicit user consent would be obtained. Finally,

establishing cross-sectional collaboration with experienced sign language educators would help further refine the system and deepen our understanding of the target audience.

Implementation Plan

Implementation Contingency

Table 23: Implementation Contingency

Risk	Identified Risk	Cause	Mitigation Strategy	Impact	Probability
Model Accuracy	Incorrect gesture prediction	Similar-looking signs or occlusion	Manual prediction confirmation, feedback collection, future retraining	High	Medium
Environmental	Poor lighting or cluttered background	Camera input degradation	User guidance, lighting recommendations	Medium	High
Performance	Frame drops or lag	Low hardware specs	Minimum system requirements defined	Medium	High
Audio Output	Missing or delayed speech	TTS cache miss or disabled speech	Cached MP3 reuse, speech toggle	Low	Low
Recording	Video recorded without audio	OpenCV video writer limitation	Store audio separately as MP3	Low	Medium
Learning Mode	Ghost model misalignment	Camera angle mismatch	Adjustable opacity and restart controls	Low	Low

Usability	User confusion with controls	Keyboard only interaction	On screen hints and report documentation	Low	Medium
System Stability	Application crash	Unhandled cases	Graceful shutdown and exception handling	High	Low

System Impact

The SignSpeak system thus bears meaningful social, educational, and technological impact, especially in the improvement of accessibility for persons who rely on Arabic Sign Language to communicate. By allowing the real-time translation of hand gestures into Arabic text and speech, this system bridges the gap in communication between the hearing-impaired and the larger community.

From an educational perspective, the integrated learning feature allows users to interactively practice and learn sign language. The visual guidance through a low-opacity hand model, complemented by real-time feedback about correctness, enables self-learning and guided practice alike. The provided feature will be of service to students, educators, family members of deaf individuals, and all people interested in learning Arabic Sign Language.

Technologically, SignSpeak demonstrates the practical usage of machine learning and computer vision in assistive systems. The offline-first design helps reduce the dependencies on network infrastructure, minimizes latency, and enhances privacy by keeping sensitive visual data on the

user's device. The modular architecture also makes the system extensible, allowing future developers to improve the model, add new features, or adapt the system to different environments and platforms.

SignSpeak, as a whole, is a positive contribution towards accessible technology by providing a combination of functionality, usability, as well as privacy. The impact of SignSpeak is not only restricted within the boundaries of real-time translation.

BIBLIOGRAPHY

[1] World Health Organization, "World Report on Hearing," *www.who.int*, Mar. 03, 2021. https://www.who.int/publications/i/item/9789240020481.

[2] J. Light and D. Mcnaughton, "Designing AAC Research and Intervention to Improve Outcomes for Individuals with Complex Communication Needs," *Augmentative and Alternative Communication*, vol. 31, no. 2, pp. 85–96, Apr. 2015, doi: https://doi.org/10.3109/07434618.2015.1036458.

[3] R. Pfau, M. Steinbach, and B. Woll, Sign Language. DE GRUYTER, 2012. doi: https://doi.org/10.1515/9783110261325.

[4] M. Ur Rehman *et al.*, "Dynamic Hand Gesture Recognition Using 3D-CNN and LSTM Networks," *Computers, Materials & Continua*, vol. 70, no. 3, pp. 4675–4690, 2022, doi: https://doi.org/10.32604/cmc.2022.019586.

[5] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, May 2012, Available:

https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

[6] A. Alazemi, A. Khandoker, and H. A. Jalab, “Arabic Sign Language Recognition using Deep Learning: A Comparative Study of CNN and Transformer Models,” arXiv preprint arXiv:2410.00681, Oct. 2024. [Online]. Available: https://arxiv.org/abs/2410.00681

[7] M. Alfarrarjeh, S. Aksoy, M. Shah, and A. Al-Hmouz, “ArSL2018: A large-scale video-based Arabic sign language dataset,” in Proc. IEEE Winter Conf. Applications of Computer Vision (WACV), 2018, pp. 147–156.

[8] S. J. Pan and Q. Yang, “A survey on transfer learning,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, Art. no. 10, 2010, doi: https://doi.org/10.1109/TKDE.2009.191.

[10] Y. Ren *et al.*, “FastSpeech 2: Fast and High-Quality End-to-End Text to Speech,” May 2021, doi: https://doi.org/10.48550/arxiv.2006.04558.

[11] A. Alazemi, A. Khandoker, and H. A. Jalab, “Arabic Sign Language Recognition using Deep Learning: A Comparative Study of CNN and Transformer Models,” arXiv preprint arXiv:2410.00681, Oct. 2024. [Online]. Available: https://arxiv.org/abs/2410.00681

[12] M. Alfarrarjeh, S. Aksoy, M. Shah, and A. Al-Hmouz, “ArSL2018: A large-scale video-based Arabic sign language dataset,” in Proc. IEEE Winter Conf. Applications of Computer Vision (WACV), 2018, pp. 147–156.

[13] A. K. Alrahayfeh and M. A. Faezipour, “RGB image-based Arabic alphabets sign language recognition,” in 2015 Long Island Systems, Applications and Technology Conference (LISAT), 2015, pp. 1–6.

[14] A. Almohimeed, M. Wald, and R. I. Damper, "Arabic Text to Arabic Sign Language Translation System for the Deaf and Hearing-Impaired Community," *Proc. 2nd Workshop on Speech and Language Processing for Assistive Technologies*, Edinburgh, Scotland, pp. 101–109, Jul. 2011.

[15] Q. Bani Baker, N. Alqudah, T. Alsmadi, and R. Awawdeh, “Image-Based Arabic Sign Language Recognition System Using Transfer Deep Learning Models,” *Applied Computational*

Intelligence and Soft Computing, vol. 2023, Art. ID 5195007, pp. 1–11, Dec. 2023. [Online]. Available: https://doi.org/10.1155/2023/5195007

[16] J. L. Lapiak, "American Sign Language and Deaf Culture," *HandSpeak*, https://www.handspeak.com/ (accessed April 25, 2025)

[17] "Direct support for people with disabilities (sign language service)," *Ministry of Education, Saudi Arabia*, https://moe.gov.sa/en/contactus/pages/spwd.aspx (accessed May 2, 2025).

[18] "SignAll Sign Language Translation App," *SignAll*, https://signall.world/ (accessed April 27, 2025).

[19] "HandSpeak: A Sign Language Dictionary Online," *Education World*. [Online]. Available: https://www.educationworld.com/awards/past/2000/r0900-05.shtmlEducation World+1MERLOT+1

[20] SignAll, "SignAll - Automatic Sign Language Translation," [Online]. Available: https://signall.world. [Accessed: May 28, 2025].

[21] A. A. Hosain, P. S. Santhalingam, P. Pathak, H. Rangwala, and J. Kosecka, "FineHand: Learning Hand Shapes for American Sign Language Recognition," *arXiv preprint arXiv:2003.08753*, 2020. [Online]. Available: https://arxiv.org/abs/2003.08753arXiv

[22] **Google**, "Hand landmarks detection guide," *Google AI Edge*, Jan. 2025. [Online]. Available: https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker

[23] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015. [Online]. Available: https://www.tensorflow.org/

[24] A. Rababaah, A. Kandil, Y. Gharib, and A. Helwa, "Visual gesture recognition for drawing applications," *Journal of Global Information Technology*, vol. 14, no. 1–2, pp. 1–9, 2019.

Appendices

Evaluation Tool

Refer to validation, verification, and testing (pages: 73-77).

Reports

Old Prototype

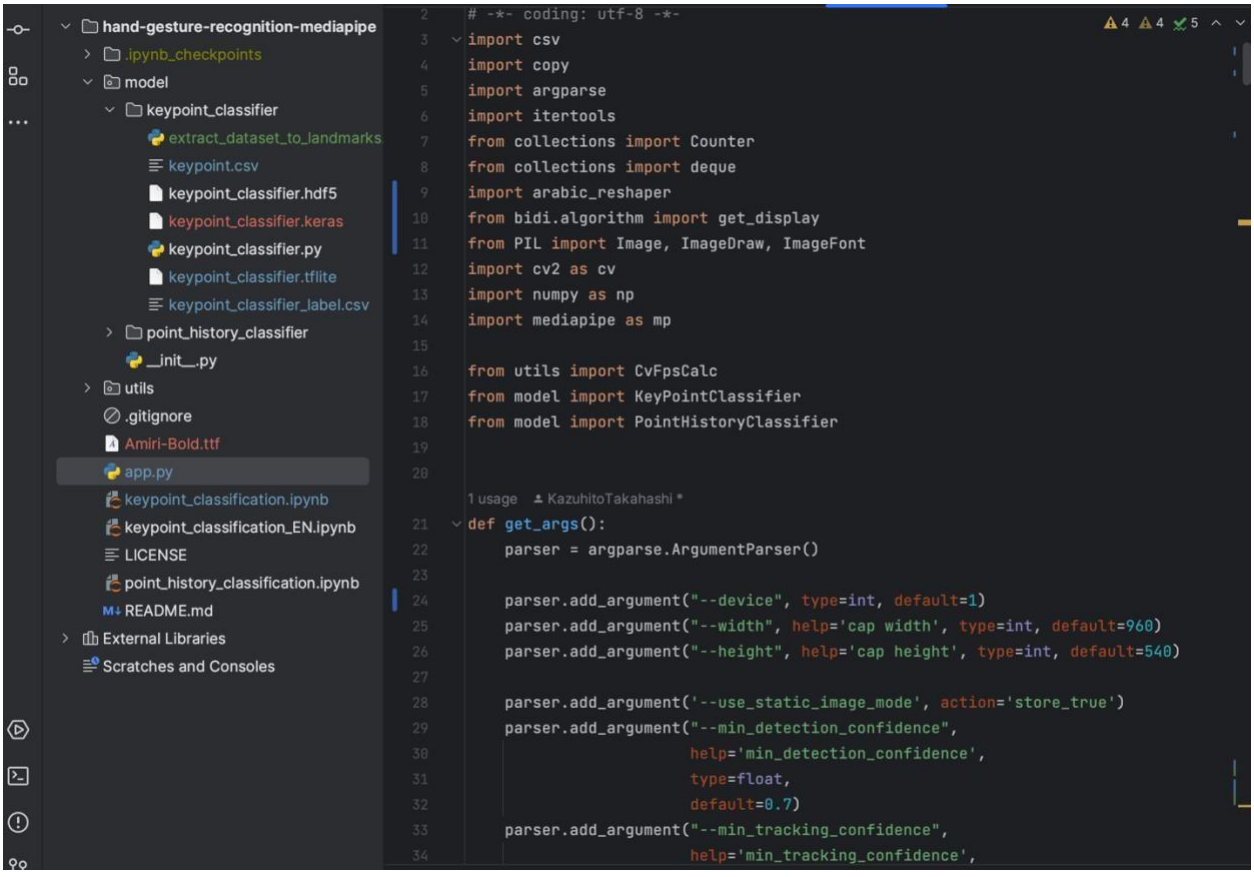
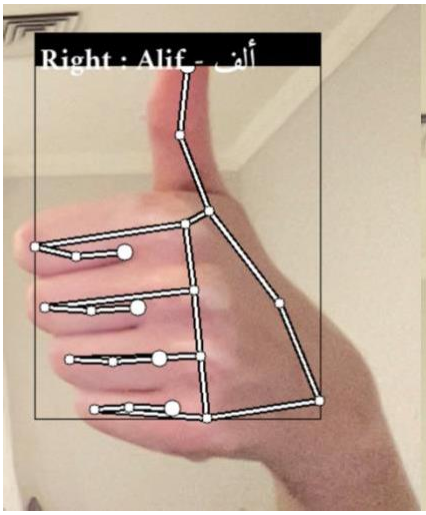


Figure 33: code



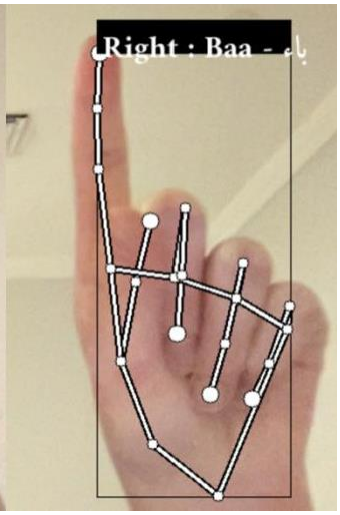


Figure 34: output for Letter Alif gesture *Figure 35: output for letter Baa gesture*

Screenshots of Current Working Prototype

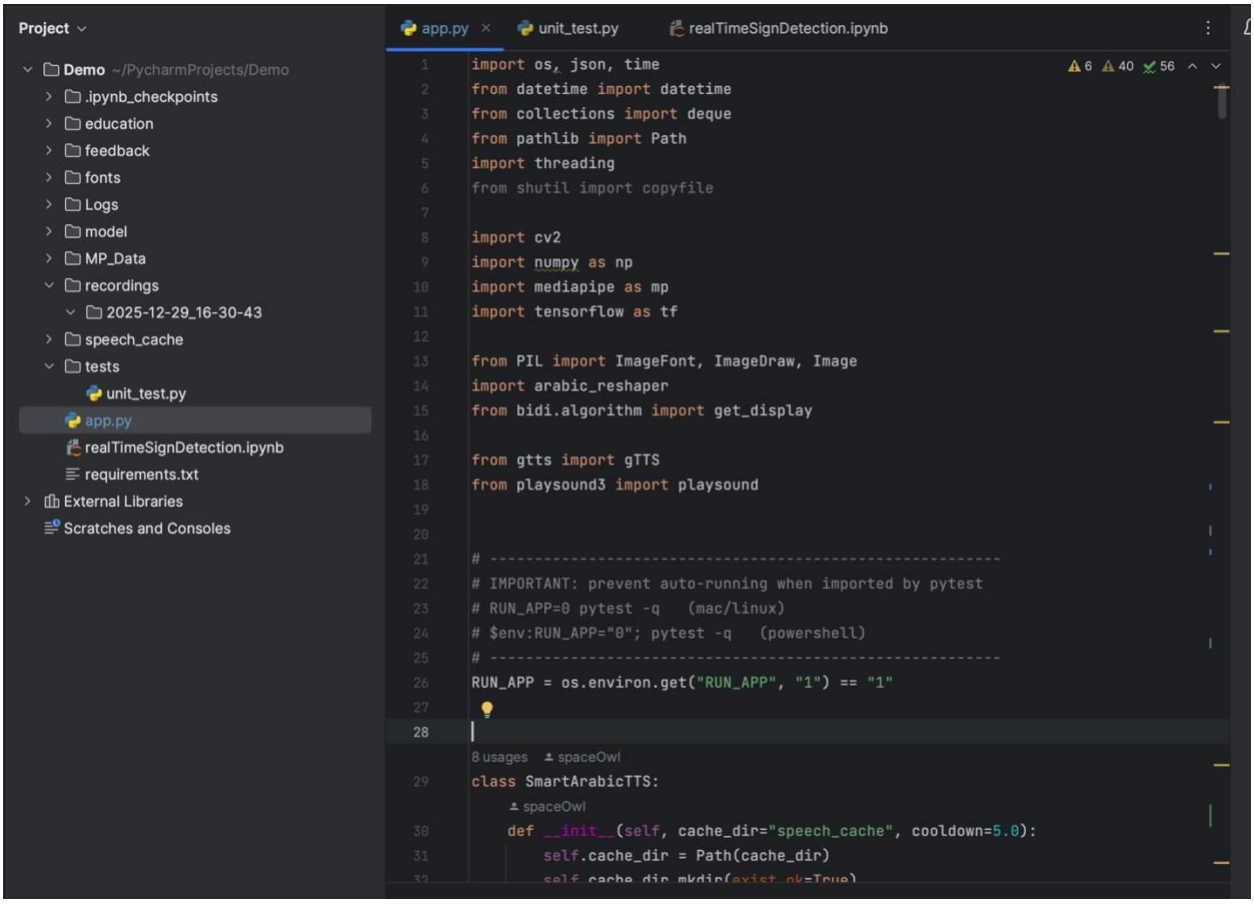
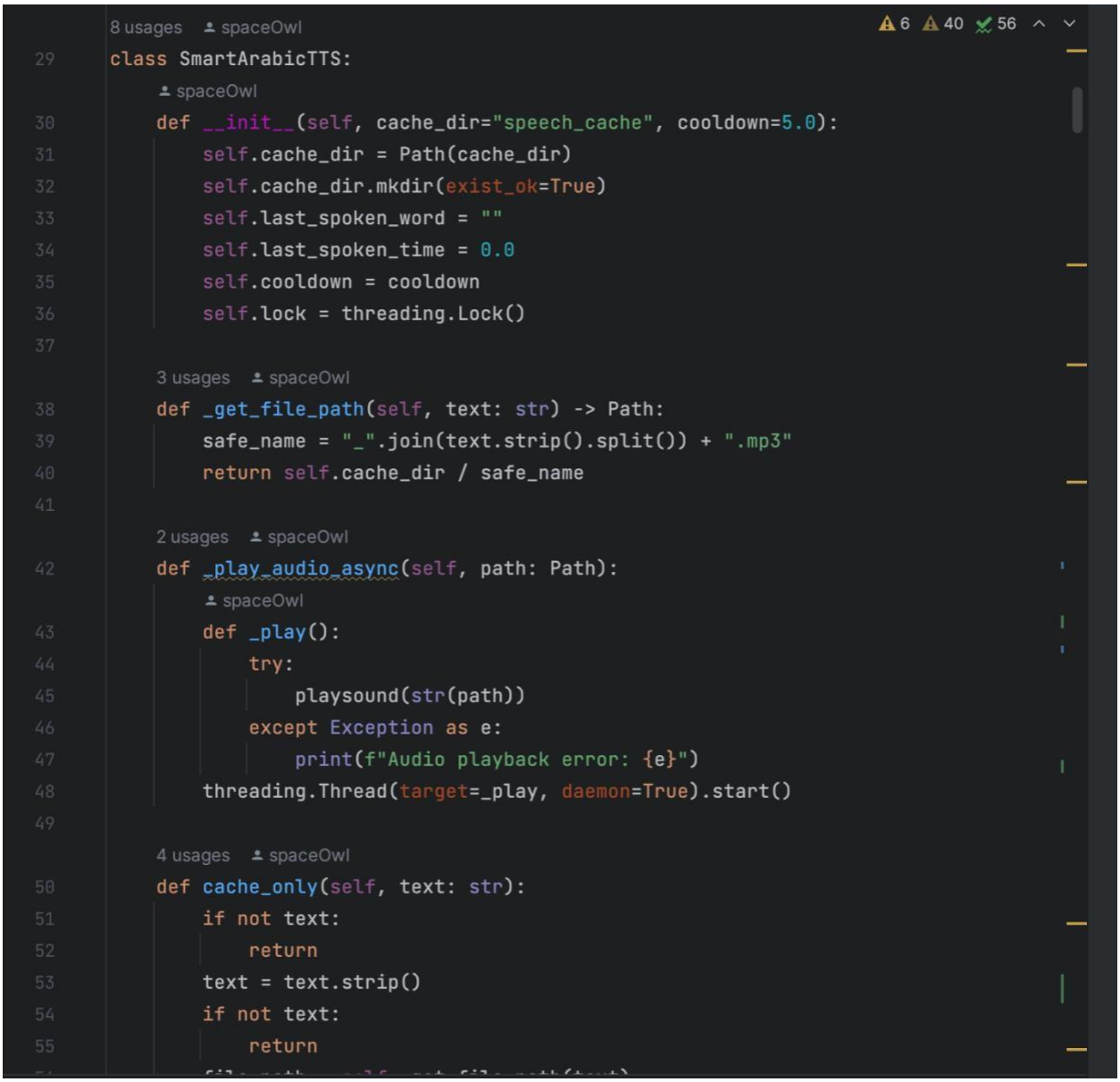


Figure 3c: Screenshots of Working Prototype



```
8 usages  spaceOwl                                                    ⚠ 6  ⚠ 40  ✓ 56
class SmartArabicTTS:
    spaceOwl
    def __init__(self, cache_dir="speech_cache", cooldown=5.0):
        self.cache_dir = Path(cache_dir)
        self.cache_dir.mkdir(exist_ok=True)
        self.last_spoken_word = ""
        self.last_spoken_time = 0.0
        self.cooldown = cooldown
        self.lock = threading.Lock()

    3 usages  spaceOwl
    def _get_file_path(self, text: str) -> Path:
        safe_name = "_".join(text.strip().split()) + ".mp3"
        return self.cache_dir / safe_name

    2 usages  spaceOwl
    def _play_audio_async(self, path: Path):
        spaceOwl
        def _play():
            try:
                playsound(str(path))
            except Exception as e:
                print(f"Audio playback error: {e}")
        threading.Thread(target=_play, daemon=True).start()

    4 usages  spaceOwl
    def cache_only(self, text: str):
        if not text:
            return
        text = text.strip()
        if not text:
            return
```

Figure 37: Screenshots of the Working Prototype

```
spaceOwl
def draw_styled_landmarks(image_bgr, results):
    mp_drawing.draw_landmarks(image_bgr, results.left_hand_landmarks, mp_holistic.HAI
    mp_drawing.draw_landmarks(image_bgr, results.right_hand_landmarks, mp_holistic.HA


spaceOwl
def extract_keypoints(results):
    lh = np.array([[p.x, p.y, p.z] for p in results.left_hand_landmarks.landmark]).f
    rh = np.array([[p.x, p.y, p.z] for p in results.right_hand_landmarks.landmark])
    return np.concatenate([lh, rh])


spaceOwl
def save_feedback_sample(seq60x252, true_label, proposed_label, conf, ok):
    ts = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
    sub = "correct" if ok else "wrong"

    label_dir = os.path.join(FEEDBACK_DIR, sub, str(true_label))
    os.makedirs(label_dir, exist_ok=True)

    npy_path = os.path.join(label_dir, f"{ts}.npy")
    np.save(npy_path, seq60x252.astype(np.float32))

    meta = {
        "timestamp": ts,
        "ok": bool(ok),
        "true_label": str(true_label),
        "proposed_label": str(proposed_label),
        "confidence": float(conf),
        "npy_path": npy_path,
    }
```

Figure 38: Screenshots of the Working Prototype

Users Guide

refer to pages 71-72

Other Relevant Documents

All the important documents will be attached in the submission alongside the source code.

Accomplished Forms

For further info regarding the system input/output/reports/forms refer to pages 55-59.

Envisioning Our Future Product

This is one of our future implementations of our product. Publishing it in the form of a website of an app to the user.

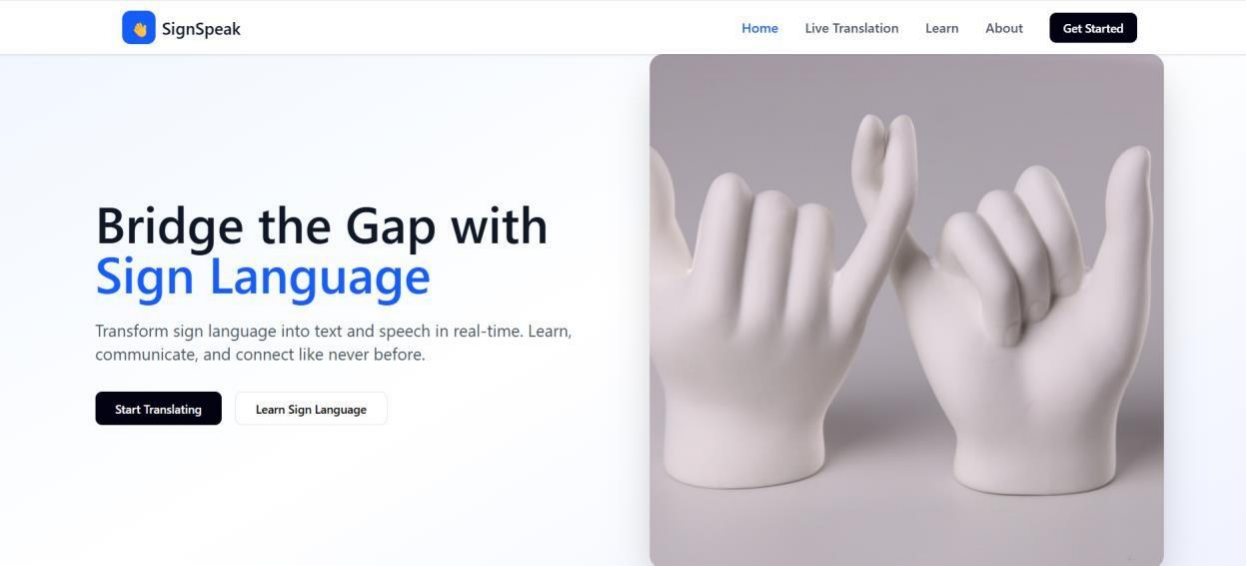


Figure 35: SignSpeak as a website (starting page)

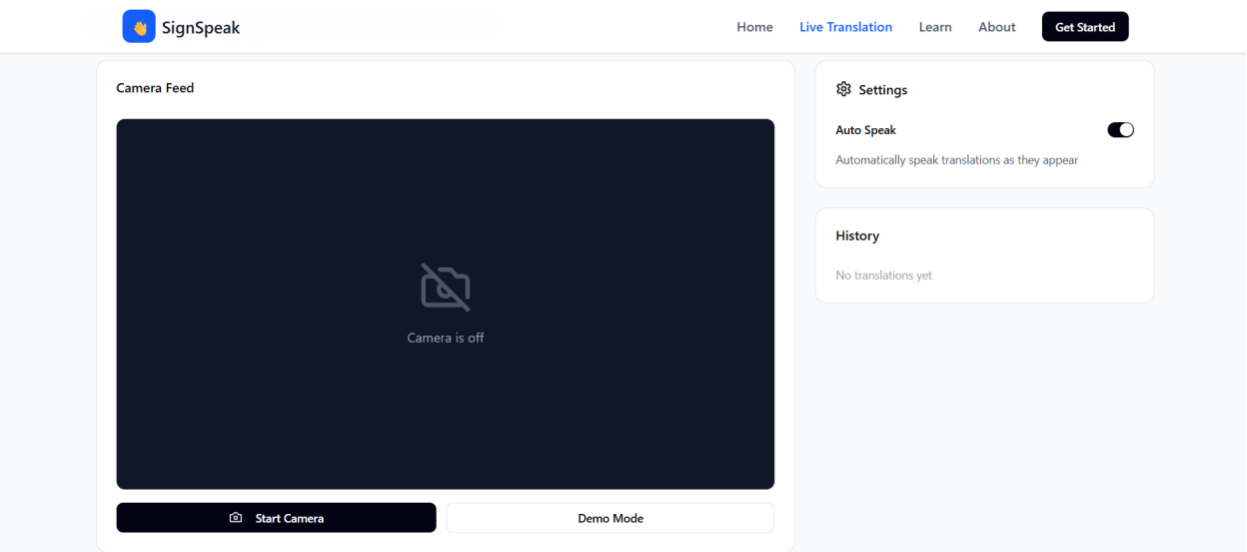


Figure 40: SignSpeak as a website (detection page)

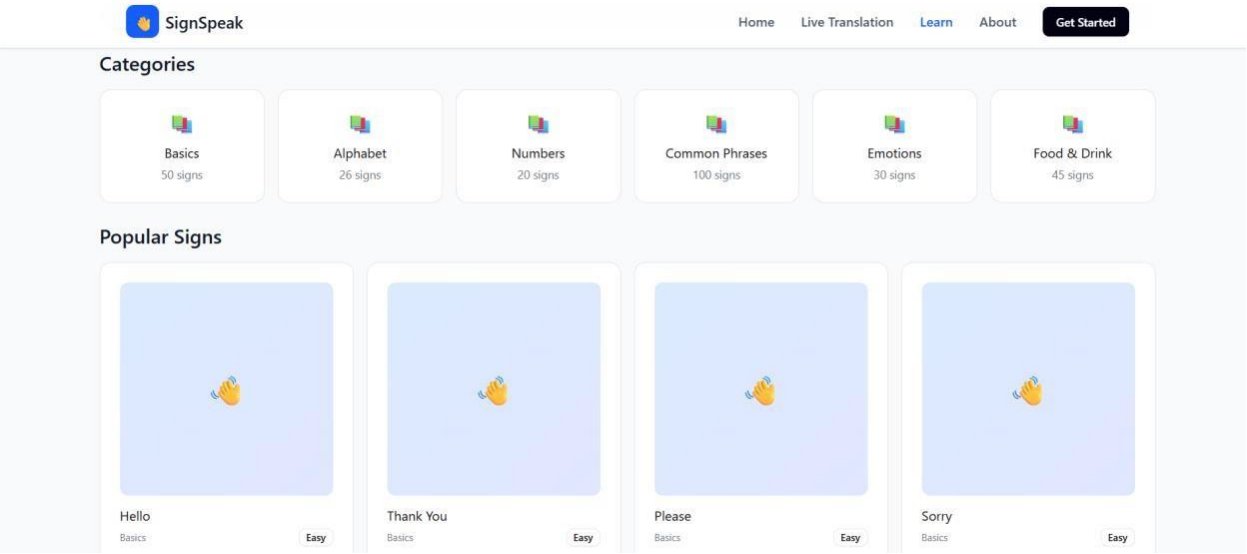


Figure 41: SignSpeak as a website (learning page)

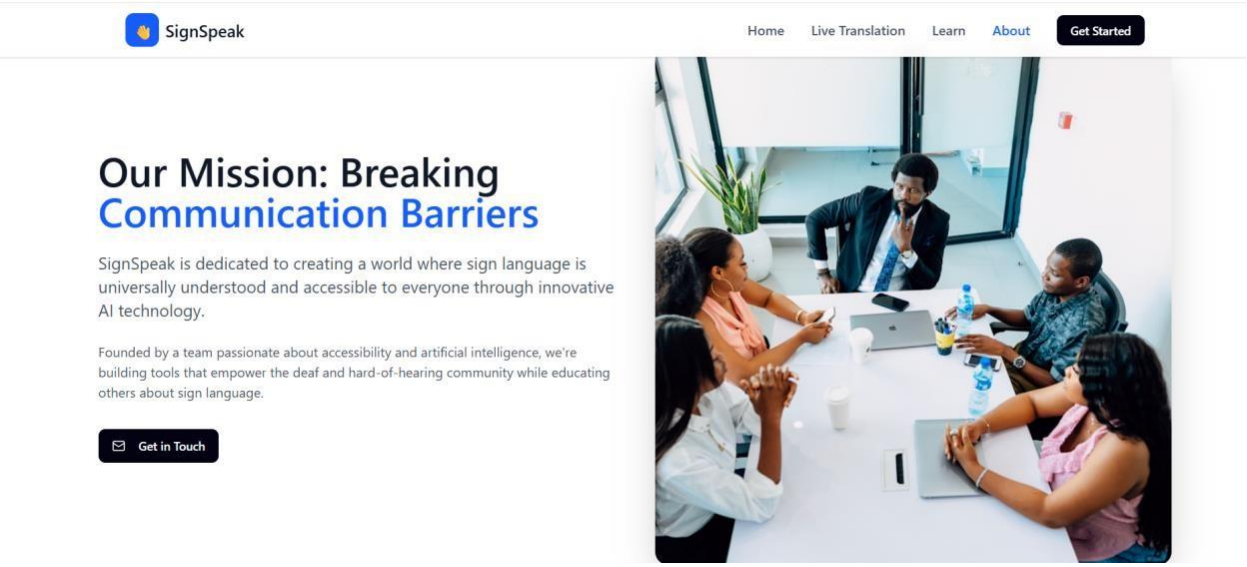


Figure 42: SignSpeak as a website (about page)

Glossary

- Augmentative and Alternative Communication (AAC): A group of strategies and tools used to help people with communication disorders
- Arabic Sign Language (ArSL): A visual language with unique grammar and syntax used by deaf and mute people in the Arab world.
- Arabic-reshaper: A specialized software library that ensures Arabic text is connected and formatted correctly for digital display.
- ArASL (Arabic Alphabet Sign Language Dataset): A collection of around 50,000 labeled images used to train the system to recognize Arabic alphabets.
- ArSL2018: A video dataset of Arabic signs used to compare and evaluate the performance of models
- Convolutional Neural Network (CNN): A type of AI model designed to find patterns in visual data such as hand shapes.
- Compatibility Testing: Checking tests to ensure that the software runs correctly on different operating systems such as Windows or MacOS.
- Central Processing Unit (CPU): The main computer component responsible for running the data and the software.
- Corporate Social Responsibility (CSR): A model where government agencies or companies sponsor the prokect to help the community.
- Dataset: A collection of information used to train or test a model.
- Deep Learning: A branch of AI that uses neural networks to learn complex patterns from large data.

- Exponential Moving Average (EMA): A math based smoothing technique that prevents the translated text from flickering.
- Entity Relationship Diagram (ERD): A chart showing the logical connections between different datas.
- Fingerspelling: A method of signing individual letters, used mostly for names or terms that do not have a dedicated sign.
- Freemium Model: A plan that includes free features.
- Gantt Chart: A visual timeline used to track progress of the project.
- Graphics Processing Unit (GPU): A special processor used to speed up the training and running of the AI models.
- Google Text-to-Speech (gTTS): Library used to convert translated Arabic text into a clear spoken voice.
- Graphical User Interface (GUI): The visual part of the application that includes texts and windows for the user to interact with.
- Hand Landmarks: Specific coordinates on the hand, such as knuckles or fingertips, that the system can track.
- Human Computer Interaction (HCI): The study of how people use technology and how to design systems that are easy for humans to navigate.
- Kaggle: An online platform used to find and download datasets for training the recognition model.
- Latency: The total response time or delay between a user's movement and the computers translation.

- Load testing: The action of running the system continuously for a long period of time to ensure stability.
- MediaPipe: An open-source tool used for fast real-time tracking of hand movements.
- Manual Features (MFs): The linguistiv part of the sign language involving handshape and its positions.
- Multi-layer Perceptron (MLP): A type of neural network architecture that is used to classify hand gestures into specific words.
- Non-Manual Features (NMFs): Linguistic parts of sign langiage that do not use hands, such as facial expression and head movements.
- OpenCV: A library used to access the webcame and process video frames.
- Python: A programming language used to build software.
- Random Access Memory (RAM): The computer memory required to run the software.
- Single-Node Topology: A technical setup where all processing happens on one local computer without needing a separate server.
- Stress Testing: Testing the system under difficult conditions, such as low light or fast movements.
- TensorFlow: An open-source platform used to build, train, and run the gesture recognition model.
- Transfer Learning: The process of taking an existing trained model and fine-tuning it to learn a new skill.
- Unit Testing: Testing individual parts of the code one by one to ensure they work before combining them.
- vents the translated text from flickering.
- Entity Relationship Diagram (ERD): A chart showing the logical connections between different datas.

- Fingerspelling: A method of signing individual letters, used mostly for names or terms that do not have a dedicated sign.
- Freemium Model: A plan that includes free features.
- Gantt Chart: A visual timeline used to track progress of the project.
- Graphics Processing Unit (GPU): A special processor used to speed up the training and running of the AI models.
- Google Text-to-Speech (gTTS): Library used to convert translated Arabic text into a clear spoken voice.
- Graphical User Interface (GUI): The visual part of the application that includes texts and windows for the user to interact with.
- Hand Landmarks: Specific coordinates on the hand, such as knuckles or fingertips, that the system can track.
- Human Computer Interaction (HCI): The study of how people use technology and how to design systems that are easy for humans to navigate.
- Kaggle: An online platform used to find and download datasets for training the recognition model.
- Latency: The total response time or delay between a user's movement and the computers translation.